

UNIVERSITÀ · PREPARAZIONE ALL'ESAME ORALE

Complementi di Reti

Manuale completo di studio e laboratorio

Routing, IPv6, ARP/NDP, RIP, OSPF, BGP, SDN, data center networking, trasporto affidabile, TCP, QUIC e controllo della congestione

Edizione rielaborata e ampliata

Appunti di Cristian Alasotto · Revisione tecnica e impaginazione editoriale

Come usare questo manuale

Questo manuale riscrive e riorganizza integralmente gli appunti del corso di **Complementi di Reti** con un obiettivo preciso: permettere la preparazione dell'esame orale senza dover ricostruire i collegamenti tra argomenti sparsi. I concetti sono presentati secondo una progressione logica: prima il livello rete e gli strumenti di laboratorio, poi i protocolli di routing, quindi il livello di trasporto e il controllo della congestione.

Ogni capitolo contiene quattro tipi di richiami:

DA RICORDARE

Definizioni, proprietà o relazioni che conviene saper esporre in modo immediato.

ATTENZIONE

Errori concettuali frequenti, semplificazioni pericolose o dettagli che all'orale possono fare la differenza.

DOMANDA TIPICA D'ESAME

Una possibile formulazione del professore, accompagnata dagli elementi che dovrebbero comparire nella risposta.

OSSERVAZIONE IN KATHARÁ

Comandi, output e fenomeni concretamente osservabili in un laboratorio Linux/FRRouting.

Metodo di studio consigliato

1. Leggere il capitolo cercando di ricostruire il problema che il protocollo vuole risolvere.
2. Ripetere ad alta voce le definizioni evidenziate nei box "Da ricordare".
3. Eseguire almeno una volta i comandi di laboratorio indicati.
4. Rispondere senza consultare il testo alle domande d'esame presenti nelle tabelle finali.
5. Confrontare protocolli simili: **RIP vs OSPF**, **ARP vs NDP**, **OSPF vs BGP**, **flow control vs congestion control**, **Go-Back-N vs Selective Repeat**, **TCP vs QUIC**.

Indice generale

- **Come usare questo manuale**
 - Metodo di studio consigliato
- **1. Livello rete: inoltro, instradamento e tabelle**
 - 1.1 Perché esiste il livello rete
 - 1.2 Forwarding e routing
 - 1.3 Host, router e forwarding IP
 - 1.4 Longest Prefix Match
 - 1.5 Metriche e criteri di selezione
 - 1.6 Famiglie di algoritmi di routing
 - 1.7 Convergenza
- **2. Ambiente di laboratorio: Kathará, Linux e FRRouting**
 - 2.1 Kathará come emulatore di rete
 - 2.2 FRRouting: architettura
 - 2.3 Avvio dei daemon
 - 2.4 La shell `vttysh`
 - 2.5 Tre tabelle da non confondere

- **3. Diagnosi e misura: ping, traceroute, cattura e generazione di traffico**

- 3.1 Metodo di diagnosi a strati
- 3.2 ping
- 3.3 traceroute
- 3.4 tcpdump
- 3.5 Wireshark
- 3.6 iperf3
- 3.7 Linux Traffic Control: tc
- 3.8 Tabella riassuntiva degli strumenti

- **4. RIP: routing Distance Vector**

- 4.1 Obiettivo e caratteristiche
- 4.2 Aggiornamento Distance Vector
- 4.3 Count-to-infinity
- 4.4 Tecniche anti-loop
- 4.5 RIPv1 e RIPv2
- 4.6 Configurazione FRR in Kathará
- 4.7 Scheda finale: RIP

- **5. IPv6, Neighbor Discovery e ARP**

- 5.1 Perché IPv6
- 5.2 Rappresentazione degli indirizzi
- 5.3 Tipi di indirizzo IPv6
- 5.4 Prefissi e subnet
- 5.5 Configurazione Linux e Kathará
- 5.6 Neighbor Discovery Protocol (NDP)
- 5.7 Address Resolution Protocol (ARP)
- 5.8 Cache ARP e Neighbor Table
- 5.9 Gratuitous ARP e Proxy ARP
- 5.10 ARP vs NDP
- 5.11 Schede finali

- **6. OSPF: routing Link State**

- 6.1 Obiettivo e modello
- 6.2 Router ID, vicini e adiacenze
- 6.3 I cinque tipi di pacchetto OSPF
- 6.4 Flooding e LSDB
- 6.5 Metrica OSPF
- 6.6 DR e BDR nelle reti multi-accesso
- 6.7 Aree e routing gerarchico
- 6.8 Tipi di LSA OSPFv2
- 6.9 Tipi di percorso
- 6.10 ECMP
- 6.11 Stub network e passive interface
- 6.12 Timer e convergenza
- 6.13 Configurazione OSPF in FRR
- 6.14 OSPF vs RIP
- 6.15 Scheda finale: OSPF

- **7. Control plane, SDN e gestione della rete**

- 7.1 Data plane e control plane
- 7.2 Software Defined Networking
- 7.3 OpenFlow e match-action
- 7.4 Traffic Engineering
- 7.5 SNMP
- 7.6 NETCONF e YANG
- 7.7 Telemetria e automazione
- 7.8 Confronto

- **8. BGP e routing inter-dominio**
 - 8.1 Internet come insieme di Sistemi Autonomi
 - 8.2 Relazioni economiche e no-free-transit
 - 8.3 Stub e multi-homing
 - 8.4 Hot Potato Routing
 - 8.5 eBGP e iBGP
 - 8.6 Finite State Machine
 - 8.7 NLRI e attributi
 - 8.8 Attributi principali
 - 8.9 Processo di selezione del best path
 - 8.10 iBGP e problema del full mesh
 - 8.11 Next-hop-self
 - 8.12 Filtri: prefix-list, AS-path list e route-map
 - 8.13 Configurazione FRR: eBGP semplice
 - 8.14 Default route per una stub AS
 - 8.15 Multi-homing attivo/passivo
 - 8.16 Load sharing e prefissi più specifici
 - 8.17 Sicurezza BGP
 - 8.18 OSPF vs BGP
 - 8.19 Scheda finale: BGP
- **9. BGP nei data center, Clos, Fat-Tree ed ECMP**
 - 9.1 Perché le topologie tradizionali non bastano
 - 9.2 ECMP
 - 9.3 Perché usare BGP come underlay
 - 9.4 ASN privati
 - 9.5 BGP multipath
 - 9.6 Convergenza rapida
 - 9.7 Traceroute e percorsi multipli
 - 9.8 Laboratorio concettuale
 - 9.9 Tabella riassuntiva
- **10. Livello di trasporto e trasferimento affidabile**
 - 10.1 Perché serve il livello di trasporto
 - 10.2 Multiplexing e demultiplexing
 - 10.3 UDP
 - 10.4 TCP
 - 10.5 Costruire teoricamente un protocollo affidabile
 - 10.6 Stop-and-Wait e utilizzazione
 - 10.7 Pipelining e sliding window
 - 10.8 Go-Back-N
 - 10.9 Selective Repeat
 - 10.10 Go-Back-N vs Selective Repeat
 - 10.11 TCP come protocollo affidabile reale
 - 10.12 Three-Way Handshake
 - 10.13 Flow control
 - 10.14 RTT e Retransmission Timeout
 - 10.15 Schede finali: UDP e TCP
- **11. Controllo della congestione: Reno, CUBIC e QUIC**
 - 11.1 Congestione e collasso
 - 11.2 Congestion Window e self-clocking
 - 11.3 Slow Start
 - 11.4 Congestion Avoidance
 - 11.5 Timeout e ACK duplicati
 - 11.6 Reno, Tahoe e AIMD
 - 11.7 Limiti RTT di AIMD
 - 11.8 TCP CUBIC
 - 11.9 BBR
 - 11.10 QUIC
 - 11.11 Vantaggi di QUIC
 - 11.12 TCP vs QUIC
 - 11.13 Test di congestion control con `iperf3` e `tc`
 - 11.14 Scheda finale: QUIC

- **12. Fairness, AQM, ECN e isolamento del traffico**
 - 12.1 Efficienza ed equità
 - 12.2 Max-Min Fairness
 - 12.3 Proportional Fairness
 - 12.4 Utility Fairness
 - 12.5 AIMD e convergenza
 - 12.6 Buffer, tail drop e bufferbloat
 - 12.7 Active Queue Management
 - 12.8 ECN
 - 12.9 DCTCP
 - 12.10 Applicazioni TCP-friendly
 - 12.11 Class-Based Queuing e schedulazione
 - 12.12 Fairness non significa sempre uguaglianza
 - 12.13 Tabella comparativa
- **13. Collegamenti trasversali tra i capitoli**
 - 13.1 Dalla risoluzione del vicino al routing globale
 - 13.2 OSPF e BGP
 - 13.3 OSPF, BFD e convergenza
 - 13.4 ECMP e trasporto
 - 13.5 Code, RTT e protocolli di routing
 - 13.6 LPM e traffic engineering BGP
- **Appendice A. Schede dei protocolli complementari**
 - A.1 ICMP e ICMPv6
 - A.2 BFD
 - A.3 SNMP
 - A.4 NETCONF/YANG
- **Appendice B. Ricettario di laboratorio Kathará**
 - B.1 Verifica iniziale di un nodo
 - B.2 Verifica del forwarding
 - B.3 Tabella di routing Linux
 - B.4 Diagnosi FRR
 - B.5 Catture per protocollo
 - B.6 Test di percorso
 - B.7 Test di prestazioni ripetibile
 - B.8 Errori Kathará frequenti
- **Appendice C. Errori concettuali da evitare**
- **Appendice D. Domande per l'esame orale**
 - D.1 Livello rete
 - D.2 FRR e Kathará
 - D.3 Diagnostica
 - D.4 RIP
 - D.5 IPv6, ARP e NDP
 - D.6 OSPF
 - D.7 SDN e gestione
 - D.8 BGP
 - D.9 Data center
 - D.10 Trasporto e affidabilità
 - D.11 Congestione, QUIC e fairness
- **Appendice E. Glossario essenziale**
- **Checklist finale prima dell'esame**

1. Livello rete: inoltro, instradamento e tabelle

1.1 Perché esiste il livello rete

Il livello rete nasce per interconnettere dispositivi appartenenti a reti locali differenti senza richiedere un'unica rete fisica omogenea. Ethernet, Wi-Fi e altre tecnologie di collegamento possono convivere perché sopra di esse opera un livello logico comune: **IP**, con un proprio schema di indirizzamento e con dispositivi intermedi, i **router**.

Il livello rete separa due problemi diversi:

- **inoltrare** velocemente un singolo pacchetto già arrivato a un router;
- **calcolare** quali percorsi debbano essere usati dall'insieme dei router.

Questa distinzione conduce ai concetti di **data plane** e **control plane**.

1.2 Forwarding e routing

Forwarding: la decisione locale

Il **forwarding** (inoltro) è il processo locale eseguito dal singolo router per ogni pacchetto. Il router:

1. riceve un frame su un'interfaccia;
2. estrae il pacchetto IP;
3. legge l'indirizzo IP di destinazione;
4. consulta la **Forwarding Information Base** (FIB), cioè la tabella ottimizzata usata dal data plane;
5. seleziona l'interfaccia di uscita e il next hop;
6. ricrea l'incapsulamento di livello collegamento e trasmette il pacchetto.

Il forwarding deve essere estremamente rapido: nei router hardware viene spesso implementato tramite ASIC e memorie specializzate; in Linux viene eseguito dal kernel.

Routing: il calcolo distribuito dei percorsi

Il **routing** (instradamento) è il processo con cui i router apprendono la raggiungibilità delle reti, calcolano percorsi e popolano le proprie tabelle. Può essere:

- **statico**, quando le rotte sono inserite manualmente dall'amministratore;
- **dinamico**, quando protocolli come RIP, OSPF o BGP scambiano informazioni e adattano i percorsi ai cambiamenti della topologia.

Il risultato dei protocolli di routing viene inserito nella **Routing Information Base** (RIB). Il sistema seleziona poi le rotte migliori e le installa nella FIB del kernel.

DA RICORDARE ROUTING

decide quali percorsi esistono e quali sono preferibili; **forwarding** applica quella decisione a ogni pacchetto. Il primo appartiene al control plane, il secondo al data plane.

DOMANDA TIPICA D'ESAME

"Qual è la differenza tra routing e forwarding?" Una risposta completa deve citare scala globale vs decisione locale, RIB/FIB, control plane/data plane e il fatto che il forwarding opera per pacchetto.

1.3 Host, router e forwarding IP

Un host possiede una tabella di routing e compie scelte elementari: decide se la destinazione è locale, se usare il loopback oppure se inviare il traffico a un gateway. Un router, invece, è configurato per **inoltare pacchetti tra interfacce diverse**.

La presenza di più interfacce non basta, da sola, a definire un router: un host multihomed può avere più schede ma non inoltrare traffico. In Linux la funzione è controllata, per IPv4, dal parametro:

```
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
```

Per IPv6 si usa il corrispondente parametro `net.ipv6.conf.all.forwarding`.

Inoltro diretto e indiretto

Un host confronta il prefisso della destinazione con i prefissi delle proprie interfacce:

- **inoltro diretto**: la destinazione appartiene a una rete direttamente connessa; l'host risolve l'indirizzo di livello 2 del destinatario e invia il frame direttamente;
- **inoltro indiretto**: la destinazione non è locale; il frame viene inviato al MAC del **next hop**, normalmente il default gateway. L'indirizzo IP di destinazione rimane quello dell'host remoto.

ATTENZIONE

Quando si invia un pacchetto a una rete remota, ARP non cerca il MAC dell'host remoto: cerca il MAC del gateway locale. Gli indirizzi IP identificano gli estremi logici; gli indirizzi MAC cambiano a ogni collegamento attraversato.

1.4 Longest Prefix Match

Una tabella può contenere più rotte compatibili con lo stesso indirizzo. Il router applica il **Longest Prefix Match (LPM)**: vince la rotta con il prefisso più lungo, cioè la più specifica.

Esempio:

Prefisso	Next hop
0.0.0.0/0	R1
10.0.0.0/8	R2
10.20.0.0/16	R3
10.20.30.0/24	R4

Un pacchetto per `10.20.30.55` usa R4, perché `/24` è la corrispondenza più specifica. Un pacchetto per `10.99.1.2` usa R2. Un indirizzo non coperto dalle altre rotte usa la default route.

LPM rende possibile l'**aggregazione dei prefissi**: molte reti contigue possono essere annunciate con un unico prefisso meno specifico, riducendo la dimensione delle tabelle.

1.5 Metriche e criteri di selezione

Una rotta non viene scelta solo perché esiste. I protocolli attribuiscono un costo ai percorsi. Le metriche possono dipendere da:

- numero di hop;
- banda nominale dei collegamenti;
- ritardo;
- affidabilità o carico;
- valori amministrativi configurati manualmente;
- policy economiche e organizzative.

È utile distinguere:

- **metrica del protocollo:** confronta rotte apprese dallo stesso protocollo, per esempio hop count in RIP o cost in OSPF;
- **distanza amministrativa:** criterio locale con cui un router preferisce una sorgente di routing rispetto a un'altra. Il valore e il nome dipendono dall'implementazione.

ERRORE COMUNE

“Il percorso migliore è sempre quello fisicamente più corto” è falso. In OSPF vince il costo configurato; in BGP le policy possono prevalere nettamente su latenza e numero di hop.

1.6 Famiglie di algoritmi di routing

Distance Vector

Ogni router conosce il costo verso i propri vicini e riceve da essi stime delle distanze verso le destinazioni. L'aggiornamento segue l'idea dell'equazione di Bellman-Ford:

$$[D_x(y) = \min_v \{ c(x,v) + D_v(y) \}]$$

Il router x sceglie il vicino v che minimizza il costo del link $c(x,v)$ più la distanza dichiarata dal vicino verso y .

Vantaggi: semplicità, basso fabbisogno di memoria, configurazione immediata.

Limiti: conoscenza parziale, convergenza potenzialmente lenta e rischio di loop transitori. RIP è l'esempio classico.

Link State

Ogni router descrive lo stato dei propri collegamenti e diffonde queste informazioni a tutto il dominio. Tutti costruiscono un database topologico coerente e calcolano localmente l'albero dei cammini minimi con Dijkstra.

Vantaggi: convergenza rapida, visione completa, metriche flessibili.

Limiti: maggiore complessità, uso di CPU e memoria, necessità di progettazione gerarchica. OSPF appartiene a questa famiglia.

Path Vector

Ogni annuncio include il percorso dei domini amministrativi attraversati. In BGP l'attributo `AS_PATH` permette di rilevare i loop: se un AS vede il proprio numero nel percorso, rifiuta la rotta.

Il Path Vector non cerca necessariamente il cammino “più veloce”: la scelta incorpora policy commerciali e operative.

Flooding

Il flooding non è una famiglia di protocolli di routing equivalente alle precedenti, ma una **tecnica di disseminazione**: un'informazione ricevuta viene inoltrata su più collegamenti, con meccanismi per evitare duplicazioni infinite. OSPF usa flooding affidabile per distribuire gli LSA.

1.7 Convergenza

Una rete è **convergente** quando tutti i router hanno una visione coerente della raggiungibilità e le loro scelte non cambiano più in assenza di nuovi eventi.

Dopo un guasto si distinguono più tempi:

1. rilevazione del guasto;
2. propagazione dell'informazione;
3. ricalcolo dei percorsi;
4. installazione delle nuove rotte;
5. ripresa del traffico.

Una convergenza lenta può generare black hole, loop, percorsi subottimali e perdita di pacchetti.

DOMANDA TIPICA D'ESAME

“Confronti Distance Vector e Link State.” Conviene strutturare la risposta su: conoscenza locale/globale, Bellman-Ford/Dijkstra, modalità di scambio, convergenza, loop, scalabilità e protocolli di esempio.

2. Ambiente di laboratorio: Kathará, Linux e FRRouting

2.1 Kathará come emulatore di rete

Kathará crea laboratori composti da container Linux collegati a domini di collisione virtuali. Ogni dispositivo può comportarsi da host, router, server o appliance di rete. Il vantaggio didattico è che si osservano gli stessi strumenti e le stesse tabelle di un sistema Linux reale, ma in una topologia riproducibile.

Una struttura tipica di laboratorio contiene:

```
lab.conf
r1.startup
r2.startup
ha.startup
hb.startup
r1/etc/frr/daemons
r1/etc/frr/frr.conf
```

- `lab.conf` definisce collegamenti, collision domain, immagini e opzioni dei dispositivi;
- i file `*.startup` eseguono i comandi all'avvio del nodo;
- le cartelle con il nome del dispositivo vengono montate nel filesystem del container e permettono di fornire configurazioni persistenti.

Comandi essenziali:

```
kathara lstart          # avvia il laboratorio corrente
kathara lclean          # arresta e rimuove il laboratorio
kathara connect -l r1   # apre una shell su r1
kathara linfo           # mostra lo stato del laboratorio
```

OSSERVAZIONE IN KATHARÁ

Le interfacce appaiono come normali interfacce Linux (`eth0`, `eth1`, ...). Il comando `ip -br addr` permette di verificare rapidamente indirizzi e stato; `ip route` mostra la tabella del kernel.

2.2 FRRouting: architettura

FRRouting (FRR) è una suite open source che trasforma Linux in un router software avanzato. Discende dai progetti Zebra e Quagga e supporta, tra gli altri, RIP, OSPF, IS-IS e BGP.

L'architettura è modulare:

- `ripd` implementa RIP;
- `ospfd` implementa OSPFv2;
- `ospf6d` implementa OSPFv3;
- `bgpd` implementa BGP;
- `zebra` coordina le informazioni di routing e comunica con il kernel.

Ogni daemon mantiene le strutture del proprio protocollo. Zebra raccoglie le rotte candidate, effettua la selezione secondo priorità e metriche e installa le rotte scelte nella tabella Linux.

Schema concettuale:

```
ripd ----\
ospfd ----> zebra ----> RIB/FIB del kernel Linux ----> forwarding
bgpd ----/
```

DA RICORDARE

Il daemon di protocollo calcola o apprende le rotte; `zebra` è l'intermediario verso il kernel. Il traffico dati non attraversa i daemon: viene inoltrato dal kernel.

2.3 Avvio dei daemon

Il file `/etc/frr/daemons` abilita i processi da avviare. In un laboratorio può essere necessario modificare, per esempio:

```
zebra=yes
ripd=yes
ospfd=yes
bgpd=yes
```

Quindi:

```
systemctl start frr
systemctl status frr
```

Per controllare i processi:

```
ps aux | grep -E 'zebra|ripd|ospfd|bgpd'
```

Problemi frequenti:

- daemon non abilitato in `/etc/frr/daemons`;
- configurazione con sintassi errata;
- interfacce non ancora configurate quando FRR parte;
- servizio avviato prima della creazione delle rotte o degli indirizzi necessari;
- file non montato nella cartella corretta del dispositivo Kathará.

2.4 La shell `vtysh`

`vtysh` fornisce una CLI unificata per i daemon FRR. Le modalità principali sono:

Modalità	Prompt tipico	Funzione
User EXEC	<code>r1></code>	Comandi di visualizzazione limitati
Privileged EXEC	<code>r1#</code>	Diagnostica completa e accesso alla configurazione
Global configuration	<code>r1(config)#</code>	Modifica della configurazione
Protocol/interface	<code>r1(config-router)#</code>	Parametri specifici

Sequenza tipica:

```
r1> enable
r1# configure terminal
r1(config)# router ospf
r1(config-router)# ...
r1(config-router)# end
r1# write memory
```

La configurazione è **operazionale**: un comando modifica lo stato corrente. Per rimuoverlo si usa normalmente lo stesso comando preceduto da `no`.

Esempio:

```
router ospf
network 10.0.0.0/24 area 0
no network 10.0.0.0/24 area 0
```

Comandi di uso quotidiano:

Comando	Scopo
<code>?</code>	Mostra i completamenti disponibili nel contesto corrente
<code>show running-config</code>	Mostra la configurazione attiva
<code>show ip route</code>	Mostra la RIB IPv4 vista da FRR
<code>show ipv6 route</code>	Mostra la RIB IPv6
<code>show interface</code>	Stato e parametri delle interfacce
<code>show logging</code>	Informazioni di log
<code>write memory</code>	Salva la configurazione
<code>exit, end</code>	Risale di un livello o torna alla modalità privilegiata

ATTENZIONE

`write memory` salva la configurazione di FRR, ma non sostituisce i file `*.startup` del laboratorio. Se il container viene ricreato senza montare il file di configurazione salvato, le modifiche possono andare perse.

2.5 Tre tabelle da non confondere

In un laboratorio FRR possono comparire informazioni diverse:

1. **Database del protocollo:** per esempio LSDB OSPF o BGP table.
2. **RIB di FRR:** rotte selezionate dai daemon e da zebra (`show ip route`).
3. **Tabella del kernel:** rotte realmente installate in Linux (`ip route`).

Una rotta può essere presente nel database BGP ma non essere selezionata; può essere selezionata da BGP ma non installata perché un'altra sorgente è preferita; può essere assente dal kernel per un next hop irraggiungibile.

DOMANDA TIPICA D'ESAME

“Che ruolo ha zebra?” Rispondere che coordina i daemon, riceve le rotte candidate, interagisce con la RIB/FIB del kernel e non inoltra direttamente i pacchetti.

3. Diagnosi e misura: ping, traceroute, cattura e generazione di traffico

3.1 Metodo di diagnosi a strati

Quando un esercizio fallisce, evitare di cambiare configurazioni a caso. Procedere dal basso verso l'alto:

1. **interfacce**: sono `UP`? gli indirizzi sono corretti?
2. **livello 2**: ARP/NDP risolve il vicino? il collision domain è corretto?
3. **routing**: esiste una rotta per la destinazione e per il percorso di ritorno?
4. **forwarding**: il router inoltra i pacchetti?
5. **daemon FRR**: è avviato? il peering è formato? le rotte sono apprese?
6. **policy**: una prefix-list, route-map o regola di firewall blocca il traffico?
7. **prestazioni**: latenza, perdita, jitter, banda e code sono coerenti?

Comandi iniziali:

```
ip -br link
ip -br addr
ip route
ip neigh
ss -lntup
```

3.2 ping

Scopo

`ping` verifica la raggiungibilità IP e misura il **Round Trip Time (RTT)** usando messaggi ICMP Echo Request ed Echo Reply.

Sintassi:

```
ping <destinazione>
ping -c 4 10.0.0.2
ping -I eth0 -c 4 10.0.0.2
ping -6 -c 4 2001:db8:1::2
```

Opzioni utili:

Opzione	Significato
<code>-c N</code>	Invia N richieste e termina
<code>-i S</code>	Intervallo tra richieste
<code>-W S</code>	Timeout di attesa della risposta
<code>-I IFACE/IP</code>	Forza interfaccia o indirizzo sorgente
<code>-s BYTES</code>	Dimensione del payload ICMP
<code>-M do</code>	Impone Don't Fragment e consente test di Path MTU in IPv4
<code>-n</code>	Evita risoluzioni DNS

Esempio di output:

```
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.42 ms
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max/mdev = 0.38/0.41/0.45/0.03 ms
```

Interpretazione:

- `icmp_seq` : numero progressivo della richiesta;
- `ttl` : TTL residuo nella risposta, utile solo come indizio sul numero di hop;
- `time` : RTT della singola richiesta;
- `packet loss` : quota di richieste senza risposta;
- `min/avg/max/mdev` : distribuzione dei tempi.

Errori comuni:

- `Destination Host Unreachable` : spesso il nodo locale o un router non riesce a risolvere il next hop o non possiede una rotta;
- `Network is unreachable` : manca una rotta nella tabella locale;
- timeout senza messaggi: pacchetti o risposte sono filtrati, il percorso di ritorno manca, oppure la destinazione è spenta;
- ping funzionante ma applicazione non funzionante: ICMP raggiunge l'host, ma porta, daemon o firewall applicativo possono essere errati.

OSSERVAZIONE IN KATHARÁ

Il primo ping può essere più lento perché deve essere preceduto da ARP o NDP. Con una cattura si osserva prima la risoluzione del vicino e poi ICMP.

ATTENZIONE

Un ping fallito non prova automaticamente che la rete sia guasta: ICMP può essere filtrato. Un ping riuscito prova la raggiungibilità IP bidirezionale per quei messaggi, non la disponibilità di ogni servizio.

3.3 traceroute

Principio di funzionamento

`traceroute` scopre i router attraversati inviando sonde con TTL/Hop Limit crescente:

1. la prima sonda ha TTL 1;
2. il primo router decrementa il TTL a zero, scarta il pacchetto e invia ICMP Time Exceeded;
3. la sonda successiva usa TTL 2 e raggiunge il secondo router;
4. il processo continua fino alla destinazione.

La modalità predefinita su Linux usa spesso sonde UDP verso porte elevate; `traceroute -I` usa ICMP Echo; `traceroute -T` usa TCP SYN.

Sintassi:

```
traceroute -n 10.0.2.2
traceroute -I -n 10.0.2.2
traceroute -T -p 443 -n 10.0.2.2
tracert -n 10.0.2.2
```

Interpretazione:

```
1  10.0.0.2  0.42 ms  0.31 ms  0.29 ms
2  10.0.1.2  0.77 ms  0.69 ms  0.72 ms
3  10.0.2.2  1.10 ms  1.02 ms  1.05 ms
```

Ogni riga rappresenta un TTL; le tre misure derivano normalmente da tre sonde. Un asterisco indica mancata risposta entro il timeout, non necessariamente perdita del traffico dati.

Errori comuni:

- risultati diversi tra esecuzioni: ECMP può distribuire sonde su percorsi diversi;
- ultimo hop non visibile: la destinazione o un firewall non invia la risposta attesa;

- hop con * * * seguito da altri hop: il router intermedio inoltra il traffico ma limita o filtra ICMP;
- loop apparente: può essere reale, ma anche un effetto di NAT, bilanciamento o identificazione delle interfacce.

DOMANDA TIPICA D'ESAME

“Perché traceroute invia pacchetti in quell’ordine?” Spiegare TTL crescente, ICMP Time Exceeded, risposta finale e differenza tra sonde UDP, ICMP e TCP.

3.4 tcpdump

Scopo e sintassi

`tcpdump` cattura pacchetti da un’interfaccia e applica filtri BPF.

```
tcpdump -nni eth0
tcpdump -nni eth0 arp
tcpdump -nni eth0 icmp
tcpdump -nni eth0 'tcp port 179'
tcpdump -nni any 'host 10.0.0.2 and (icmp or arp)'
```

Opzioni fondamentali:

Opzione	Effetto
<code>-i eth0</code>	Seleziona l’interfaccia
<code>-n</code>	Non risolve nomi DNS
<code>-nn</code>	Non risolve nemmeno nomi di porte
<code>-e</code>	Mostra header Ethernet e MAC
<code>-t, -tttt</code>	Modifica il formato del timestamp
<code>-v, -vv, -vvv</code>	Aumenta il dettaglio
<code>-s 0</code>	Cattura l’intero pacchetto
<code>-c N</code>	Termina dopo N pacchetti
<code>-w file.pcap</code>	Salva in formato pcap
<code>-r file.pcap</code>	Legge una cattura

Il comando spesso usato nei laboratori:

```
tcpdump -tenni eth1
```

combina: nessun timestamp (`-t`), header Ethernet (`-e`), output numerico (`-nn`) e interfaccia `eth1` (`-i eth1`).

Filtri utili

```
tcpdump -nni eth0 'src host 10.0.0.1'
tcpdump -nni eth0 'dst net 10.0.2.0/24'
tcpdump -nni eth0 'udp port 520'           # RIP
tcpdump -nni eth0 'ip proto 89'           # OSPFv2
tcpdump -nni eth0 'tcp port 179'          # BGP
tcpdump -nni eth0 'icmp6'                 # NDP e altri ICMPv6
```

Errori comuni:

- catturare sull’interfaccia sbagliata;
- dimenticare `-n` e confondere ritardi DNS con ritardi di rete;
- usare un filtro troppo restrittivo;

- aspettarsi di vedere il traffico di un altro collision domain;
- non considerare offloading del kernel, che può alterare l'aspetto dei pacchetti catturati sull'host.

3.5 Wireshark

Wireshark analizza graficamente le catture. In Kathará può essere eseguito come container dedicato e collegato al collision domain da osservare, oppure si può salvare un `.pcap` con tcpdump e aprirlo sul computer host.

Filtri di visualizzazione utili:

```
arp
icmp
icmpv6
rip
ospf
bgp
tcp.port == 179
ip.addr == 10.0.0.1
tcp.analysis.retransmission
```

Distinguere:

- **capture filter**: limita ciò che viene registrato, usa sintassi BPF (`tcp port 179`);
- **display filter**: filtra dopo la cattura, usa sintassi Wireshark (`tcp.port == 179`).

Elementi da leggere:

1. lista dei pacchetti;
2. albero dei protocolli;
3. bytes grezzi;
4. flussi ricostruiti con *Follow TCP Stream*;
5. grafici RTT, throughput e sequenze TCP.

OSSERVAZIONE IN KATHARÁ

Per verificare che il container Wireshark osservi il traffico corretto, collega esplicitamente la sua interfaccia allo stesso collision domain dei nodi interessati e filtra per gli indirizzi del laboratorio. Traffico estraneo può provenire dal container stesso o da servizi di gestione.

3.6 iperf3

Modello client-server

`iperf3` genera traffico sintetico e misura throughput, ritrasmissioni, perdita e jitter.

Server:

```
iperf3 -s
iperf3 -s -D          # daemon
iperf3 -s -l          # una sola sessione e poi termina
```

Client:

```
iperf3 -c 10.0.0.2
iperf3 -c 10.0.0.2 -t 30
iperf3 -c 10.0.0.2 -R
```

Il server usa per impostazione predefinita la porta 5201. Nei test UDP rimane una connessione di controllo TCP e viene generato il flusso dati UDP.

Test TCP

```
iperf3 -c 10.0.0.2 -t 20 -P 4
iperf3 -c 10.0.0.2 -w 256K
iperf3 -c 10.0.0.2 -C cubic
```

- **-P 4** : quattro flussi paralleli, utili quando un singolo flusso non riempie il collegamento;
- **-w** : dimensione richiesta dei socket buffer; non equivale in modo perfetto alla congestion window e il kernel può modificarla;
- **-C** : seleziona l'algoritmo di congestion control, se disponibile nel kernel;
- **-R** : inverte la direzione del test;
- **--bidir** : testa entrambe le direzioni simultaneamente.

Il **Bandwidth-Delay Product** è:

[BDP = banda;[bit/s] RTT;[s]]

Per ottenere i byte da mantenere “in volo” si divide per 8. Un collegamento da 1 Gbit/s con RTT 40 ms ha un BDP di circa 5 MB. Buffer o finestre molto inferiori possono impedire a un singolo flusso di sfruttare la banda.

Output TCP:

```
[ 5] 0.00-10.00 sec 1.08 GBytes 928 Mbits/sec 12 sender
[ 5] 0.00-10.00 sec 1.08 GBytes 927 Mbits/sec receiver
```

Le colonne principali sono intervallo, dati trasferiti, bitrate e, lato sender, ritrasmissioni. Molte ritrasmissioni possono indicare perdita, code, MTU errata o sovraccarico.

Test UDP

```
iperf3 -c 10.0.0.2 -u -b 20M -t 20
iperf3 -c 10.0.0.2 -u -b 100M -l 1200
```

L'utente sceglie il bitrate target con **-b**. L'output include:

- **jitter**: variazione del ritardo di arrivo, non il ritardo assoluto;
- **lost/total datagrams**: pacchetti persi;
- **bitrate ricevuto**.

Non esiste una soglia universale di jitter accettabile: dipende dal codec, dal buffer di playout e dall'applicazione.

JSON e automazione

```
iperf3 -c 10.0.0.2 -J > result.json
jq '.end.sum_received.bits_per_second' result.json
```

Questo consente test ripetibili e grafici temporali.

Errori comuni:

- server non avviato o porta 5201 non raggiungibile;
- test nella direzione sbagliata;
- confondere Mbit/s e MB/s;
- attribuire ogni limite alla rete ignorando CPU, namespace, virtualizzazione o singolo flusso TCP;
- usare UDP a bitrate eccessivo e interpretare la perdita come guasto anziché come saturazione intenzionale.

3.7 Linux Traffic Control: `tc`

Architettura

`tc` controlla le code associate alle interfacce:

- **qdisc**: disciplina di accodamento;
- **class**: suddivisione gerarchica della banda nelle qdisc classful;
- **filter**: regola che classifica il traffico e lo assegna a una classe;
- **action**: operazione applicata, per esempio drop, police o redirect.

Per ispezionare:

```
tc qdisc show dev eth0
tc -s qdisc show dev eth0
tc class show dev eth0
tc filter show dev eth0
```

Per rimuovere la qdisc root:

```
tc qdisc del dev eth0 root
```

Se non esiste, il comando può restituire un errore innocuo.

TBF: limite rigido di banda

```
tc qdisc add dev eth0 root tbf \
    rate 10mbit burst 32kbit latency 400ms
```

Il Token Bucket Filter accumula token a una velocità stabilita. Un pacchetto può essere trasmesso se sono disponibili token sufficienti. `burst` permette brevi raffiche; `latency` limita il tempo massimo di permanenza.

Verifica:

```
iperf3 -s # su hb
iperf3 -c <IP_HB> -t 20 # su ha
```

Il throughput dovrebbe avvicinarsi al limite, tenendo conto di overhead e misurazione.

HTB: gerarchie e garanzie

HTB permette classi con banda garantita (`rate`) e tetto massimo (`ceil`). Esempio concettuale:

```
tc qdisc add dev eth0 root handle 1: htb default 30
tc class add dev eth0 parent 1: classid 1:1 htb rate 100mbit
tc class add dev eth0 parent 1:1 classid 1:10 htb rate 20mbit ceil 100mbit
tc class add dev eth0 parent 1:1 classid 1:30 htb rate 5mbit ceil 100mbit
```

Le classi possono prendere banda non utilizzata fino al `ceil` , secondo le regole della gerarchia.

`netem`: emulare una rete imperfetta

```
tc qdisc add dev eth0 root netem delay 100ms
tc qdisc change dev eth0 root netem delay 100ms 20ms

tc qdisc change dev eth0 root netem loss 1%
tc qdisc change dev eth0 root netem delay 50ms loss 0.5%
```

`netem` può introdurre ritardo, jitter, perdita, duplicazione, riordinamento e corruzione. È ideale per confrontare protocolli e applicazioni in condizioni controllate.

Per rimuovere:

```
tc qdisc del dev eth0 root
```

fq_codel e bufferbloat

`fq_codel` combina **Fair Queuing** e **Controlled Delay**. Separa i flussi in code e controlla il tempo di permanenza dei pacchetti, riducendo il bufferbloat. È molto diffuso nei sistemi Linux, ma la qdisc predefinita dipende dalla distribuzione e dalla configurazione.

```
tc qdisc replace dev eth0 root fq_codel
```

Traffico in ingresso

Il traffico può essere limitato con la qdisc `ingress` e un'azione `police`, che scarta ciò che supera la soglia. Il policing non modella una coda: è brusco e induce TCP a ridurre la velocità tramite perdite.

Per un vero shaping in ingresso si usa spesso un dispositivo **IFB**, verso cui si reindirige il traffico per applicare una qdisc di uscita.

Trappole operative

- Le regole `tc` sono volatili e vanno ricreate al riavvio.
- Più qdisc root non possono coesistere sulla stessa interfaccia.
- Applicare `netem` sul lato sbagliato modifica una direzione diversa da quella prevista.
- Ritardo e perdita possono essere asimmetrici.
- `tc` agisce nel namespace del container: verificare di essere sul nodo corretto.

ESPERIMENTO COMPLETO

1. misura il baseline con iperf3; 2) applica TBF a 10 Mbit/s; 3) ripeti il test; 4) aggiungi 50 ms di delay con netem; 5) osserva con ping l'aumento dell'RTT e con iperf3 l'effetto sul singolo flusso TCP; 6) rimuovi le qdisc e verifica il ripristino.

3.8 Tabella riassuntiva degli strumenti

Strumento	Scopo	Output chiave	Errore tipico	Uso in Kathará
<code>ping</code>	Raggiungibilità e RTT	loss, min/avg/max	Dimenticare il percorso di ritorno	Verificare hop-by-hop
<code>traceroute</code>	Percorso e hop	router, RTT per hop	Interpretare * come guasto certo	Individuare il punto di arresto
<code>tcpdump</code>	Cattura testuale	header e timestamp	Interfaccia/filtro errati	Vedere ARP, OSPF, BGP, ICMP
Wireshark	Analisi grafica	dissezione e flussi	Confondere capture/display filter	Studiare handshake e protocolli
<code>iperf3</code>	Throughput, loss, jitter	bitrate, retransmit, jitter	MB/s vs Mbit/s	Validare link e policy
<code>tc</code>	Gestione delle code	statistiche qdisc	Regole residue o direzione errata	Riprodurre colli di bottiglia
<code>netem</code>	Emulazione di difetti	delay/loss configurati	Applicazione sul nodo sbagliato	Testare convergenza e protocolli

4. RIP: routing Distance Vector

4.1 Obiettivo e caratteristiche

RIP (Routing Information Protocol) è un Interior Gateway Protocol basato su Distance Vector. È semplice da configurare e utile in reti piccole o in laboratori didattici, ma non è adatto a topologie estese.

RIPv2 usa:

- UDP porta 520;
- multicast `224.0.0.9` per gli aggiornamenti IPv4;
- **hop count** come metrica;
- valore massimo valido 15 hop;
- metrica 16 per indicare destinazione irraggiungibile.

La scelta del percorso considera il numero di router attraversati, non banda o latenza. Due collegamenti molto diversi possono quindi apparire equivalenti.

4.2 Aggiornamento Distance Vector

Ogni router mantiene una tabella con:

- prefisso di destinazione;
- next hop;
- metrica;
- interfaccia di uscita;
- timer associati.

Periodicamente invia ai vicini il proprio vettore delle distanze. Quando riceve un aggiornamento, aggiunge il costo del collegamento verso il vicino e confronta la nuova stima con quella esistente.

Esempio:

```
R1 --1-- R2 --1-- R3  
R2 annuncia a R1: "R3 è a distanza 1"  
R1 calcola: costo(R1,R2) 1 + distanza dichiarata 1 = 2
```

Il processo è distribuito e iterativo. La rete converge quando gli aggiornamenti non modificano più le tabelle.

4.3 Count-to-infinity

Supponiamo che R2 raggiunga la rete X direttamente e che R3 la raggiunga tramite R2. Se il collegamento R2-X cade:

1. R2 elimina la rotta diretta;
2. R3 potrebbe continuare ad annunciare X con metrica 2, perché non ha ancora elaborato il guasto;
3. R2 interpreta l'annuncio di R3 come un percorso alternativo;
4. R2 e R3 si indicano reciprocamente come next hop;
5. la metrica cresce a ogni aggiornamento fino a raggiungere l'infinito di RIP, cioè 16.

Questo è il problema del **count-to-infinity**: le buone notizie si propagano rapidamente, mentre le cattive possono richiedere più cicli.

DA RICORDARE

In RIP "infinito" non è un valore matematico illimitato: è la metrica 16. Proprio questo limite impedisce che il conteggio continui indefinitamente, ma limita la rete a 15 hop.

4.4 Tecniche anti-loop

Split Horizon

Un router non annuncia su un'interfaccia una rotta appresa dalla stessa interfaccia. Se R3 ha imparato X da R2, non la riannuncia a R2.

Poisoned Reverse

Il router riannuncia la rotta verso il vicino da cui l'ha appresa, ma con metrica 16. Il messaggio esplicito impedisce al vicino di usare quel router come percorso alternativo.

Route Poisoning

Quando una rotta diventa irraggiungibile, viene annunciata immediatamente con metrica 16.

Triggered Update

Il router non aspetta il prossimo aggiornamento periodico: invia subito la variazione.

Hold-down e timer

Timer di invalidazione e garbage collection evitano di reinserire troppo rapidamente informazioni instabili. I valori dipendono dall'implementazione, ma il principio è mantenere temporaneamente lo stato della rotta per impedire oscillazioni.

ATTENZIONE

Split horizon e poisoned reverse riducono i loop semplici tra due router, ma non eliminano ogni possibile loop in topologie più complesse. La convergenza di un Distance Vector resta un problema distribuito.

4.5 RIPv1 e RIPv2

Caratteristica	RIPv1	RIPv2
Classless	No	Sì
Maschera nei messaggi	No	Sì
VLSM/CIDR	Non supportati correttamente	Supportati
Invio aggiornamenti	Broadcast	Multicast 224.0.0.9
Autenticazione	No	Prevista
Metrica	Hop count	Hop count

RIPv2 mantiene la semplicità del protocollo ma risolve i principali limiti di indirizzamento classful di RIPv1.

4.6 Configurazione FRR in Kathará

Topologia minima:

```
ha --- R1 --- R2 --- hb
```

Su ogni router:

```
systemctl start frr
vtysh
```

Configurazione di R1:

```
configure terminal
router rip
version 2
network 10.0.0.0/24
network 10.0.1.0/24
end
write memory
```

Configurazione di R2:

```
configure terminal
router rip
version 2
network 10.0.1.0/24
network 10.0.2.0/24
end
write memory
```

Verifica:

```
show ip rip status
show ip rip
show ip route rip
show ip route
```

Cattura:

```
tcpdump -nni eth0 'udp port 520'
```

Si osservano pacchetti RIPv2 destinati a `224.0.0.9`.

redistribute connected

Normalmente il comando `network` abilita RIP sulle interfacce corrispondenti e rende annunciabili le reti connesse. `redistribute connected` importa nel processo RIP tutte le rotte connesse presenti nella RIB, comprese quelle di interfacce su cui RIP non è attivo.

```
router rip
redistribute connected
```

Usarlo senza filtri può annunciare reti indesiderate, incluse reti di gestione.

Default route per una stub network

Una rete con un solo punto di uscita non necessita dell'intera tabella esterna. Il router di bordo può originare una default route nel dominio RIP:

```
router rip
default-information originate
```

Prima deve esistere una reale uscita coerente, per esempio una rotta statica:

```
ip route add default via 192.0.2.1
```

In alternativa si può redistribuire una rotta statica con opportuna policy. La scelta dipende dal laboratorio e dalla versione di FRR.

ESPERIMENTO DI CONVERGENZA

Avvia un ping continuo tra gli host, interrompi un collegamento con `ip link set ethX down` e osserva: perdita dei pacchetti, aggiornamenti RIP con `tcpdump`, variazioni in `show ip route` e tempo necessario alla convergenza.

4.7 Scheda finale: RIP

Voce	Sintesi
Scopo	Routing dinamico interno semplice in reti piccole
Algoritmo	Distance Vector / Bellman-Ford distribuito
Metrica	Hop count; 16 = infinito
Vantaggi	Configurazione semplice, basso overhead concettuale, utile per didattica
Svantaggi	Limite di 15 hop, convergenza lenta, metriche poco espressive, loop transitori
Trasporto	UDP 520
Invio RIPv2	Multicast 224.0.0.9
Casi d'uso	Piccole reti, ambienti legacy, laboratori
Domande d'orale	Count-to-infinity; split horizon vs poisoned reverse; RIPv1 vs RIPv2; significato della convergenza; perché 16 è infinito

5. IPv6, Neighbor Discovery e ARP

5.1 Perché IPv6

IPv4 usa indirizzi a 32 bit e offre circa 4,3 miliardi di combinazioni. La crescita di Internet ha richiesto tecniche come NAT, assegnazione dinamica e riuso degli indirizzi privati. IPv6 amplia lo spazio a **128 bit**, cioè circa $(3,4^{\wedge}\{38\})$ indirizzi.

L'abbondanza di indirizzi riduce la necessità del NAT come strumento di conservazione. Non significa però che IPv6 renda inutili firewall, segmentazione o policy di sicurezza. Esiste anche NAT66, ma non rappresenta il modello architetturale preferito.

Altri obiettivi di IPv6:

- autoconfigurazione più evoluta;
- semplificazione dell'header base;
- uso esteso del multicast;
- eliminazione del broadcast;
- supporto nativo a meccanismi di estensione tramite extension header.

5.2 Rappresentazione degli indirizzi

Un indirizzo IPv6 è composto da otto gruppi esadecimali da 16 bit:

```
2001:0db8:0000:0000:0000:ff00:0042:8329
```

Regole di compressione:

1. gli zeri iniziali di ogni gruppo possono essere omessi;
2. una sequenza contigua di gruppi `0000` può essere sostituita da `::` una sola volta.

Risultato:

```
2001:db8::ff00:42:8329
```

`::` può comparire una sola volta perché altrimenti la ricostruzione del numero di gruppi sarebbe ambigua.

DOMANDA TIPICA D'ESAME

"Espanda l'indirizzo `2001:db8::1`." Risposta: `2001:0db8:0000:0000:0000:0000:0000:0001`.

5.3 Tipi di indirizzo IPv6

Unicast

Identifica una singola interfaccia.

- **Global Unicast**: normalmente prefissi instradabili globalmente, tipicamente nell'intervallo `2000::/3`.
- **Link-Local**: prefisso `fe80::/10`; valido solo sul collegamento locale e non inoltrato dai router. Ogni interfaccia IPv6 ne possiede normalmente uno.
- **Unique Local Address (ULA)**: `fc00::/7`, usato per indirizzamento interno non destinato al routing globale.
- **Loopback**: `::1/128`.
- **Unspecified**: `::/128`.

Multicast

Gli indirizzi multicast iniziano con `ff00::/8`. Un pacchetto è consegnato alle interfacce appartenenti al gruppo.

Esempi:

- `ff02::1`: tutti i nodi sul link;
- `ff02::2`: tutti i router sul link;
- gruppi solicited-node: usati da NDP per la risoluzione dei vicini.

Anycast

Lo stesso indirizzo unicast è configurato su più nodi. Il routing consegna il traffico all'istanza topologicamente più vicina secondo le metriche disponibili. Anycast è comune per DNS, CDN e servizi distribuiti.

ERRORE COMUNE

IPv6 non usa broadcast. Le funzioni che in IPv4 ricorrono al broadcast vengono realizzate tramite multicast mirato, per esempio NDP con i gruppi solicited-node.

5.4 Prefissi e subnet

La notazione CIDR resta invariata:

```
2001:db8:10::/64
```

Nelle LAN IPv6 il prefisso `/64` è il caso standard per SLAAC e per molte funzioni del protocollo. I primi 64 bit identificano la subnet; i restanti 64 formano l'Interface Identifier.

Non bisogna applicare automaticamente a IPv6 le abitudini di subnetting IPv4: lo spazio è progettato per una gerarchia ampia e per subnet numerose.

5.5 Configurazione Linux e Kathará

```
ip -6 addr add 2001:db8:1::1/64 dev eth0
ip -6 addr show dev eth0
ip -6 route show
ip -6 route add default via 2001:db8:1::2 dev eth0
ping -6 2001:db8:1::2
```

Il link-local del next hop può richiedere l'indicazione dell'interfaccia:

```
ping -6 fe80::1%eth0
ip -6 route add default via fe80::1 dev eth0
```

Il simbolo `%eth0` identifica lo scope, necessario perché lo stesso indirizzo link-local potrebbe esistere su collegamenti diversi.

5.6 Neighbor Discovery Protocol (NDP)

In IPv6 ARP non esiste. Le funzioni di scoperta sono integrate nel **Neighbor Discovery Protocol**, basato su ICMPv6 e multicast.

NDP svolge più compiti:

- risoluzione indirizzo IPv6 -> indirizzo di livello 2;
- scoperta dei router;
- scoperta dei prefissi;
- determinazione dei parametri del link;
- Neighbor Unreachability Detection (NUD);
- Duplicate Address Detection (DAD);
- redirect;
- supporto a SLAAC.

Messaggi principali:

Messaggio ICMPv6	Funzione
Router Solicitation (RS)	Un host chiede ai router di annunciarsi
Router Advertisement (RA)	Il router annuncia prefissi, gateway e parametri
Neighbor Solicitation (NS)	Risoluzione del vicino, DAD o verifica raggiungibilità
Neighbor Advertisement (NA)	Risposta o annuncio di informazioni sul vicino
Redirect	Suggerisce un next hop migliore sullo stesso link

Risoluzione del vicino

Per conoscere il MAC associato a `2001:db8:1::2`, l'host non invia una richiesta a tutti. Costruisce il relativo **solicited-node multicast address** e invia una Neighbor Solicitation solo al gruppo pertinente. Il destinatario risponde con Neighbor Advertisement.

Questo riduce il numero di nodi che devono elaborare la richiesta rispetto al broadcast ARP.

SLAAC

Con **Stateless Address Autoconfiguration** il router invia RA contenenti il prefisso. L'host costruisce autonomamente un indirizzo, verifica con DAD che non sia duplicato e installa i parametri di rete.

SLAAC non implica necessariamente un indirizzo pubblico: dipende dal prefisso annunciato. Inoltre DNS e altre informazioni possono essere fornite tramite RA o DHCPv6, secondo la configurazione.

Tabella dei vicini IPv6

```
ip -6 neigh show
ip -6 neigh flush dev eth0
```

Gli stati possono includere `REACHABLE`, `STALE`, `DELAY`, `PROBE`, `INCOMPLETE`, `FAILED`.

Cattura:

```
tcpdump -nni eth0 icmp6
```

In Wireshark:

```
icmpv6.type == 135 || icmpv6.type == 136
```

ATTENZIONE

Bloccare indiscriminatamente ICMPv6 rompe funzioni essenziali di IPv6, tra cui NDP e Path MTU Discovery. ICMPv6 non è un accessorio: è parte del funzionamento del protocollo.

5.7 Address Resolution Protocol (ARP)

ARP risolve un indirizzo IPv4 in un indirizzo MAC sullo stesso dominio di livello 2.

Scenario:

```
PC-A: IP 10.0.0.1, MAC AA:AA:AA:AA:AA:AA
PC-B: IP 10.0.0.2, MAC BB:BB:BB:BB:BB:BB
```

PC-A conosce l'IP di PC-B ma deve costruire un frame Ethernet. Procede così:

1. controlla la cache dei vicini;
2. se manca l'associazione, invia una **ARP Request** in broadcast Ethernet;
3. tutti ricevono la richiesta, ma PC-B riconosce il proprio IP;

4. PC-B invia una **ARP Reply**, normalmente unicast;
5. PC-A memorizza l'associazione e trasmette il frame.

ARP Request, in forma semplificata:

```
Who has 10.0.0.2? Tell 10.0.0.1
```

ARP Reply:

```
10.0.0.2 is at BB:BB:BB:BB:BB:BB
```

5.8 Cache ARP e Neighbor Table

Linux usa il sottosistema dei vicini per IPv4 e IPv6:

```
ip neigh show  
ip neigh show dev eth0  
ip neigh flush all
```

Il comando storico:

```
arp -n
```

è ancora presente in alcuni ambienti, ma `ip neigh` è preferibile.

Le entry non sono permanenti per impostazione predefinita: attraversano stati e scadono per adattarsi a cambiamenti di interfaccia o topologia.

Esperimento ARP con `tcpdump`

Terminale 1:

```
ip neigh flush all  
tcpdump -ttni eth0 arp
```

Terminale 2:

```
ping -c 1 10.0.0.2
```

Sequenza osservabile:

1. ARP Request in broadcast;
2. ARP Reply in unicast;
3. ICMP Echo Request;
4. ICMP Echo Reply.

Ripetendo subito il ping, ARP potrebbe non comparire perché l'associazione è in cache.

5.9 Gratuitous ARP e Proxy ARP

Gratuitous ARP

Un host invia una richiesta o risposta ARP relativa al proprio indirizzo senza essere stato interrogato. Usi:

- aggiornare le cache dopo un cambio di MAC;
- aggiornare le tabelle degli switch;
- supportare failover e indirizzi virtuali;
- rilevare conflitti di indirizzo.

Non è intrinsecamente autenticato e può essere abusato in attacchi di ARP spoofing.

Proxy ARP

Un router risponde a una richiesta ARP per un indirizzo appartenente a un'altra rete, fornendo il proprio MAC. L'host invia quindi il frame al router, che inoltra il pacchetto.

Proxy ARP può facilitare scenari legacy, ma nasconde i confini di livello 3, complica la diagnosi e aumenta la superficie d'attacco. È normalmente disabilitato salvo necessità.

5.10 ARP vs NDP

Aspetto	ARP	NDP
Protocollo di rete	IPv4	IPv6
Incapsulamento	Protocollo proprio su Ethernet	ICMPv6
Richiesta	Broadcast	Multicast solicited-node
Risposta	Tipicamente unicast	Neighbor Advertisement
Scoperta router	No	Sì
Autoconfigurazione	No	Supporta SLAAC
Rilevazione raggiungibilità	Limitata	NUD integrato
DAD	Meccanismi separati	Integrato

5.11 Schede finali

IPv6

Voce	Sintesi
Scopo	Indirizzamento e inoltro scalabili a 128 bit
Vantaggi	Spazio enorme, autoconfigurazione, niente broadcast, gerarchia ampia
Svantaggi	Transizione, doppio stack, nuove procedure operative e di sicurezza
Algoritmo	Non è un algoritmo di routing; è un protocollo di livello rete
Metrica	Dipende dal protocollo di routing usato
Casi d'uso	Internet, reti enterprise, data center, IoT
Domande d'orale	Compressione; unicast/multicast/anycast; link-local; perché /64 ; ruolo di ICMPv6

ARP

Voce	Sintesi
Scopo	Risolvere IPv4 in MAC sul link locale
Vantaggi	Semplice, automatico, diffuso
Svantaggi	Broadcast, assenza di autenticazione, spoofing
Algoritmo	Request/Reply con cache
Metrica	Non applicabile
Casi d'uso	Ethernet IPv4 e reti locali
Domande d'orale	Request vs Reply; cache; ARP per destinazione remota; gratuitous e proxy ARP

NDP

Voce	Sintesi
Scopo	Scoperta vicini/router, SLAAC, DAD, NUD in IPv6
Vantaggi	Multicast mirato, funzioni integrate, autoconfigurazione
Svantaggi	Dipendenza da ICMPv6, rischi di spoofing senza protezioni
Algoritmo	Scambio RS/RA/NS/NA/Redirect
Metrica	Non applicabile
Casi d'uso	Ogni LAN IPv6
Domande d'orale	Perché sostituisce ARP; solicited-node multicast; SLAAC; DAD; stati neighbor cache

6. OSPF: routing Link State

6.1 Obiettivo e modello

OSPF (Open Shortest Path First) è un Interior Gateway Protocol Link State. Ogni router costruisce una rappresentazione della topologia, chiamata **Link State Database (LSDB)**, e calcola i percorsi con l'algoritmo SPF di Dijkstra.

Il processo concettuale è:

1. scoperta dei vicini;
2. formazione delle adiacenze;
3. sincronizzazione dei database;
4. flooding affidabile degli LSA;
5. calcolo SPF;
6. installazione delle rotte migliori.

OSPFv2 trasporta IPv4 e opera direttamente sopra IP con protocol number 89. OSPFv3 è la versione progettata per IPv6 e successive estensioni.

DA RICORDARE

RIP scambia "distanze" con i vicini; OSPF diffonde descrizioni dei link e ogni router calcola localmente l'intera mappa della propria area.

6.2 Router ID, vicini e adiacenze

Ogni router OSPF possiede un **Router ID** a 32 bit, scritto come un indirizzo IPv4 ma usato come identificatore logico. Può essere configurato manualmente; in assenza, l'implementazione lo sceglie in base alle interfacce disponibili.

I pacchetti Hello scoprono i vicini e verificano la compatibilità di parametri come:

- Area ID;
- Hello e Dead Interval;
- autenticazione, se configurata;
- opzioni e tipo di rete;
- subnet coerente sulle reti broadcast.

Se i parametri critici non coincidono, l'adiacenza non raggiunge lo stato Full.

Stati tipici del neighbor:

Down -> Init -> 2-Way -> ExStart -> Exchange -> Loading -> Full

- **2-Way** : comunicazione bidirezionale confermata;
- **ExStart/Exchange** : negoziazione e scambio dei riassunti del database;
- **Loading** : richiesta degli LSA mancanti;
- **Full** : database sincronizzati per quella adiacenza.

6.3 I cinque tipi di pacchetto OSPF

Tipo	Funzione
Hello	Scoperta e mantenimento dei vicini, elezione DR/BDR
Database Description (DBD o DD)	Riassunto del contenuto della LSDB
Link State Request (LSR)	Richiesta di LSA specifici
Link State Update (LSU)	Trasporto di uno o più LSA

Tipo	Funzione
Link State Acknowledgment (LSAck)	Conferma affidabile della ricezione

Gli LSA sono i record informativi; gli LSU sono pacchetti che trasportano LSA. Non sono sinonimi.

OSPFv2 usa gli indirizzi multicast:

- 224.0.0.5 : AllSPFRouters;
- 224.0.0.6 : AllDRouters, usato per DR e BDR sulle reti multi-accesso.

6.4 Flooding e LSDB

Ogni LSA contiene identificatori, numero di sequenza, età e checksum. Quando un router riceve una versione più recente:

1. aggiorna la LSDB;
2. conferma la ricezione;
3. propaga l'LSA sulle altre adiacenze;
4. pianifica un nuovo calcolo SPF se la topologia è cambiata.

Il flooding è affidabile ma controllato: numeri di sequenza e acknowledgment impediscono duplicazioni infinite e incoerenze.

Tutti i router della stessa area devono convergere a una LSDB coerente. Router diversi calcolano alberi SPF diversi perché ciascuno pone se stesso alla radice.

6.5 Metrica OSPF

OSPF usa il **cost**. La formula tradizionale è:

[Cost =]

Il valore della reference bandwidth dipende dalla configurazione e dall'implementazione. Se resta troppo basso, link moderni di velocità diversa possono ricevere lo stesso costo intero. Conviene configurare una reference bandwidth coerente su tutti i router.

In FRR:

```
router ospf
 auto-cost reference-bandwidth 100000
```

Il valore è espresso in Mbit/s. Un costo può anche essere impostato sull'interfaccia:

```
interface eth0
 ip ospf cost 50
```

ATTENZIONE

Il costo OSPF non è una misura dinamica del traffico istantaneo. Di norma deriva dalla banda configurata o da un valore amministrativo. OSPF non sposta automaticamente i flussi perché un link è momentaneamente congestionato.

6.6 DR e BDR nelle reti multi-accesso

Su una LAN con molti router, creare un'adiacenza Full tra ogni coppia causerebbe un numero quadratico di relazioni. OSPF elegge:

- **Designated Router (DR);**
- **Backup Designated Router (BDR);**
- gli altri diventano **DROther**.

I DROther mantengono adiacenze Full con DR e BDR, ma normalmente restano in stato 2-Way tra loro.

Criteri di elezione:

1. priorità OSPF dell'interfaccia più alta;
2. a parità, Router ID più alto.

Priorità 0 significa non eleggibile. L'elezione è non preemptive: un router arrivato dopo con priorità maggiore non sostituisce automaticamente il DR esistente.

Comandi:

```
show ip ospf neighbor
show ip ospf interface eth0
```

Configurazione:

```
interface eth0
ip ospf priority 100
```

DOMANDA TIPICA D'ESAME

“Perché esistono DR e BDR?” Citare la riduzione delle adiacenze e del flooding sulle reti broadcast; precisare che non sono necessariamente il gateway degli host e che l'elezione è per segmento.

6.7 Aree e routing gerarchico

Per scalare, OSPF divide il dominio in **aree**. Ogni router conosce in dettaglio la topologia della propria area, mentre tra aree vengono scambiate informazioni riassunte.

Area 0

L'**Area 0**, o backbone, collega le altre aree. Il traffico inter-area deve logicamente attraversare il backbone. I virtual link sono un meccanismo eccezionale di riparazione, non una buona progettazione ordinaria.

Ruoli dei router

- **Internal Router**: tutte le interfacce OSPF nella stessa area;
- **Backbone Router**: almeno un'interfaccia in Area 0;
- **Area Border Router (ABR)**: interfacce in più aree, tipicamente una in Area 0;
- **Autonomous System Boundary Router (ASBR)**: redistribuisce rotte provenienti da altri protocolli o sorgenti.

Un router può ricoprire più ruoli contemporaneamente.

Aree stub

Un'area stub riduce le informazioni esterne ricevute. L'ABR fornisce una default route e limita gli LSA esterni. Esistono varianti, alcune standard e altre dipendenti dal vendor:

- stub area;
- totally stubby area;
- NSSA (Not-So-Stubby Area), che permette una forma controllata di redistribuzione esterna.

6.8 Tipi di LSA OSPFv2

Tipo	Nome	Origine	Contenuto
1	Router LSA	Ogni router	Link e interfacce del router nell'area
2	Network LSA	DR	Router collegati a una rete multi-accesso
3	Summary / Inter-Area Prefix	ABR	Prefissi raggiungibili in altre aree

Tipo	Nome	Origine	Contenuto
4	ASBR Summary	ABR	Raggiungibilità di un ASBR
5	AS External	ASBR	Rotte esterne al dominio OSPF
7	NSSA External	ASBR in NSSA	Rotte esterne originate in NSSA, poi tradotte

Comandi:

```
show ip ospf database
show ip ospf database router
show ip ospf database network
show ip ospf database summary
show ip ospf database external
```

ERRORE COMUNE

La LSDB non è la tabella di routing. La LSDB descrive la topologia; SPF produce percorsi candidati; la RIB contiene rotte selezionate; la FIB del kernel serve al forwarding.

6.9 Tipi di percorso

Ordine concettuale di preferenza OSPF:

1. intra-area;
2. inter-area;
3. external type 1 (E1);
4. external type 2 (E2).

Per le rotte E1 il costo totale include costo interno verso l'ASBR più metrica esterna. Per E2 la metrica esterna domina; il costo interno è usato come criterio di spareggio tra rotte comparabili.

6.10 ECMP

Se OSPF calcola più percorsi con lo stesso costo, può installarli contemporaneamente: **Equal-Cost Multi-Path (ECMP)**.

In FRR:

```
show ip route 10.20.0.0/16
```

può mostrare più next hop. Zebra passa i percorsi al kernel, che distribuisce i flussi usando meccanismi di hashing. Normalmente l'hashing è per flusso, non per singolo pacchetto, per evitare riordinamenti.

6.11 Stub network e passive interface

Una LAN con soli host deve essere annunciata, ma non è utile inviare Hello alla ricerca di router inesistenti.

Soluzione:

```
router ospf
network 10.10.10.0/24 area 0
passive-interface eth1
```

L'interfaccia passiva:

- non invia Hello;
- non forma adiacenze;
- mantiene la rete come prefisso interno OSPF.

È preferibile a redistribuire la rete connessa, perché la redistribuzione la trasformerebbe in rotta esterna.

6.12 Timer e convergenza

Valori comuni sulle reti broadcast e point-to-point:

- Hello Interval: 10 s;
- Dead Interval: 40 s;
- LSRefreshTime: 30 min;
- MaxAge: 60 min.

Questi valori non sono universali e devono coincidere tra vicini dove richiesto.

La convergenza comprende:

- rilevazione del guasto;
- generazione/flooding dell'LSA;
- calcolo SPF;
- aggiornamento RIB/FIB.

Un link che passa fisicamente a down può essere rilevato subito. Un router che smette di rispondere senza abbassare il link viene rilevato dal Dead Interval, salvo meccanismi più rapidi.

BFD

Bidirectional Forwarding Detection (BFD) è un protocollo leggero di rilevazione rapida dei guasti. Può informare OSPF o BGP in tempi sub-secondo, separando il rilevamento dalla logica del protocollo di routing.

Le implementazioni moderne usano timer SPF adattivi/throttling per evitare ricalcoli continui durante eventi instabili. Non esiste un unico valore fisso valido per tutte le versioni.

6.13 Configurazione OSPF in FRR

Esempio:

```
configure terminal
router ospf
  ospf router-id 1.1.1.1
  network 10.0.0.0/24 area 0
  network 10.0.1.0/24 area 0
  passive-interface eth0
end
write memory
```

Alternativa per interfaccia:

```
interface eth1
 ip ospf area 0
```

Diagnostica:

```
show ip ospf
show ip ospf interface
show ip ospf neighbor
show ip ospf database
show ip route ospf
show ip ospf route
```

Cattura:

```
tcpdump -nni eth1 'ip proto 89'
```

Procedura quando l'adiacenza non sale

1. `ip -br addr`: indirizzi e interfacce;
2. `ping` tra gli indirizzi del link;
3. `tcpdump 'ip proto 89'`: arrivano Hello?

4. `show ip ospf interface` : area, timer, tipo rete, cost;
5. `show ip ospf neighbor` : stato;
6. confrontare Area ID, subnet, timer, autenticazione e MTU;
7. verificare che il daemon `ospfd` sia attivo.

Un blocco in `ExStart/Exchange` è spesso associato a problemi di MTU o negoziazione; un vicino assente indica più spesso connettività, daemon, area o timer.

6.14 OSPF vs RIP

Aspetto	RIP	OSPF
Famiglia	Distance Vector	Link State
Algoritmo	Bellman-Ford distribuito	Dijkstra/SPF
Conoscenza	Stime dai vicini	LSDB topologica
Metrica	Hop count	Cost
Scalabilità	Bassa	Elevata tramite aree
Convergenza	Più lenta	Generalmente più rapida
Loop	Count-to-infinity	Ridotti dalla LSDB coerente e SPF
Complessità	Bassa	Alta
Aggiornamenti	Periodici e triggered	Hello + flooding di variazioni/refresh

6.15 Scheda finale: OSPF

Voce	Sintesi
Scopo	Routing interno scalabile e rapido
Algoritmo	Link State + Dijkstra/SPF
Metrica	Cost, spesso derivato dalla banda di riferimento
Vantaggi	Convergenza rapida, aree, ECMP, metriche flessibili, assenza di count-to-infinity
Svantaggi	Configurazione complessa, LSDB e SPF richiedono risorse, progettazione delle aree
Trasporto	Direttamente su IP, protocol number 89
Casi d'uso	Enterprise, campus, ISP, reti interne medio-grandi
Domande d'orale	Cinque pacchetti; LSA vs LSU; DR/BDR; tipi di LSA; aree e ABR; E1 vs E2; passive-interface; convergenza e BFD

7. Control plane, SDN e gestione della rete

7.1 Data plane e control plane

Il livello rete può essere interpretato come l'unione di due piani funzionali:

- **Data plane:** inoltra i pacchetti secondo regole già installate. È locale, veloce e opera a frequenza per-pacchetto.
- **Control plane:** costruisce, modifica e distribuisce le regole di inoltro. Opera su tempi più lunghi e reagisce a guasti, policy e cambiamenti della topologia.

Nel modello tradizionale **per-router**, ogni dispositivo esegue localmente il proprio control plane. OSPF, RIP e BGP sono distribuiti: ogni router comunica con gli altri e costruisce una propria vista.

Nel modello **logicamente centralizzato**, tipico del Software Defined Networking, la logica è separata dagli apparati di inoltro e viene coordinata da un controller.

DA RICORDARE

“Centralizzato” in SDN significa soprattutto *logicamente* centralizzato. Il controller può essere implementato come cluster distribuito per disponibilità e scalabilità.

7.2 Software Defined Networking

Lo **SDN** nasce dall'esigenza di rendere il controllo della rete programmabile, automatizzabile e indipendente dal singolo apparato proprietario.

Architettura a livelli:

```
Applicazioni di rete
|
Northbound API
|
Controller SDN / Network Operating System
|
Southbound API
|
Switch e router programmabili
```

Piano delle applicazioni

Contiene funzioni come:

- traffic engineering;
- firewalling centralizzato;
- bilanciamento;
- monitoraggio;
- segmentazione;
- calcolo dei percorsi;
- automazione delle policy.

Controller

Mantiene una vista della topologia, raccoglie eventi e statistiche, decide le regole e le installa nei dispositivi. Esempi di concetti associati sono controller ONOS, OpenDaylight e piattaforme proprietarie.

Southbound API

Collega controller e dispositivi. **OpenFlow** è l'esempio storico più noto, ma non è l'unica possibilità. Si usano anche NETCONF, gNMI, P4Runtime e API proprietarie.

Northbound API

Esponde servizi e astrazioni alle applicazioni. Non esiste un unico standard universale: spesso si usano API REST, intent-based interface e modelli specifici.

7.3 OpenFlow e match-action

Il modello OpenFlow rappresenta l'inoltro tramite tabelle di flusso. Una regola contiene:

- campi di **match**, per esempio indirizzi, porte, protocollo, VLAN;
- priorità;
- contatori;
- azioni, come inoltro, drop, modifica header o invio al controller.

Esempio concettuale:

```
match: dst_ip = 10.0.2.0/24, tcp_dst = 443
azione: output porta 3
```

La programmabilità rende più semplice applicare policy coerenti su molti dispositivi, ma introduce requisiti di affidabilità del controller e di sicurezza del canale di controllo.

7.4 Traffic Engineering

Il **Traffic Engineering (TE)** cerca di distribuire i flussi in modo da usare meglio la capacità, evitare congestioni e rispettare vincoli.

Le metriche IGP tradizionali offrono un controllo indiretto: modificando i costi si influenzano i percorsi di tutti i flussi. SDN permette decisioni più granulari perché il controller vede la rete in modo globale e può installare regole specifiche.

Obiettivi possibili:

- minimizzare il massimo utilizzo dei link;
- garantire banda a classi prioritarie;
- evitare determinati nodi;
- ridurre latenza;
- bilanciare carico;
- rispettare policy di sicurezza.

7.5 SNMP

SNMP (Simple Network Management Protocol) permette a un manager di leggere e modificare variabili esposte dagli agenti sui dispositivi.

Componenti:

- **manager/NMS**: sistema di gestione;
- **agent**: processo sul dispositivo;
- **MIB (Management Information Base)**: struttura gerarchica degli oggetti gestiti;
- **OID (Object Identifier)**: identificatore univoco di una variabile.

Operazioni comuni:

- **GET**: legge un oggetto;
- **GETNEXT** / **GETBULK**: percorre la MIB;
- **SET**: modifica un valore, se consentito;
- **TRAP**: notifica asincrona senza richiesta;
- **INFORM**: notifica confermata.

SNMPv1 e v2c usano community string e offrono sicurezza limitata. SNMPv3 aggiunge autenticazione e cifratura.

ATTENZIONE

La MIB non è il database dei dati correnti: è il modello gerarchico che definisce gli oggetti. I valori sono mantenuti dagli agenti e letti tramite gli OID.

7.6 NETCONF e YANG

La gestione moderna separa il **modello dei dati** dal **protocollo di configurazione**.

- **YANG** descrive strutture, tipi, vincoli, configurazione e stato operativo.
- **NETCONF** trasporta operazioni di configurazione, normalmente su una sessione sicura SSH e con dati codificati in XML.

Operazioni NETCONF tipiche:

- `<get>` : legge stato e configurazione;
- `<get-config>` : legge un datastore;
- `<edit-config>` : modifica la configurazione;
- `<commit>` : applica modifiche candidate, se supportato;
- `<lock>` / `<unlock>` : coordina accessi concorrenti.

Datastore concettuali:

- `running` : configurazione attiva;
- `candidate` : configurazione preparata prima del commit;
- `startup` : configurazione di avvio, se presente.

Vantaggi rispetto alla CLI manuale:

- struttura machine-readable;
- validazione;
- transazioni;
- automazione consistente;
- gestione di molti dispositivi.

7.7 Telemetria e automazione

Il polling SNMP periodico può essere inefficiente per dati ad alta frequenza. Le piattaforme moderne usano spesso **streaming telemetry**, in cui i dispositivi pubblicano aggiornamenti a un collector.

Tecnologie associate:

- gNMI/gRPC;
- modelli YANG;
- OpenConfig;
- sistemi time-series e dashboard.

Il principio è passare da interrogazioni sporadiche a un flusso strutturato di osservazioni.

7.8 Confronto

Tecnologia	Ruolo principale	Modello dati	Modalità
CLI/vtysh	Configurazione operativa	Testo proprietario	Interattiva
SNMP	Monitoraggio e gestione semplice	MIB/OID	Polling + trap
NETCONF	Configurazione strutturata	YANG/XML	RPC transazionale
gNMI	Configurazione e telemetria	YANG/Protobuf	gRPC, streaming
OpenFlow	Programmazione del forwarding	Tabelle match-action	Controller-switch

DOMANDA TIPICA D'ESAME

“Qual è la differenza tra SNMP, NETCONF e YANG?” SNMP è un protocollo di gestione basato su MIB; NETCONF è un protocollo RPC per configurazioni strutturate; YANG è un linguaggio di modellazione, non un protocollo di trasporto.

8. BGP e routing inter-dominio

8.1 Internet come insieme di Sistemi Autonomi

Internet non è amministrata come un'unica rete. È composta da **Autonomous Systems (AS)**, insiemi di reti e router sotto un'amministrazione comune e con una policy di routing coerente.

Si distinguono:

- **Intra-AS routing**: routing interno, realizzato con IGP come OSPF o IS-IS;
- **Inter-AS routing**: scambio di raggiungibilità tra domini, realizzato con BGP.

BGP (Border Gateway Protocol) è un protocollo Path Vector. Annuncia prefissi IP accompagnati da attributi. La scelta non dipende solo dalla lunghezza del percorso: le policy economiche e operative sono centrali.

8.2 Relazioni economiche e no-free-transit

Relazioni comuni:

Provider-Customer

Il cliente paga il provider per ottenere connettività verso Internet. Il provider trasporta il traffico del cliente verso il resto della rete.

Peering

Due reti si scambiano il traffico proprio e dei rispettivi clienti, spesso senza pagamento per volume, perché entrambe ottengono un vantaggio.

Transit

Una rete consente a un'altra di raggiungere destinazioni ulteriori. Il transit ha un valore economico e non viene normalmente offerto gratuitamente a reti non clienti.

La regola **no free transit** spiega molte policy di esportazione:

- le rotte dei clienti vengono annunciate a tutti, perché trasportare quel traffico genera ricavi;
- le rotte apprese da provider o peer vengono normalmente annunciate solo ai clienti;
- non si offre transit tra due provider o tra due peer.

Questo produce il modello **valley-free**: un percorso tipico sale da customer a provider, può attraversare al massimo un collegamento peer-to-peer e poi scende da provider a customer.

DOMANDA TIPICA D'ESAME

“Perché un provider può non annunciare a un peer una rotta appresa da un altro provider?” Per evitare di fornire transit gratuito tra reti che non pagano.

8.3 Stub e multi-homing

Una **stub AS** non trasporta traffico di terzi. Può avere:

- un solo collegamento a un provider;
- più collegamenti allo stesso provider;
- collegamenti a provider diversi, cioè multi-homing.

Un cliente dual-homed non deve annunciare a un provider le rotte apprese dall'altro, altrimenti potrebbe diventare accidentalmente rete di transit.

8.4 Hot Potato Routing

Un AS di grandi dimensioni può avere più punti di uscita verso la stessa destinazione. Con **Hot Potato Routing** sceglie il punto di uscita con costo IGP interno più basso, liberandosi del traffico il prima possibile.

Questo non minimizza necessariamente il percorso end-to-end. Minimizza il costo interno all'AS che prende la decisione.

È l'esempio classico dell'interazione tra BGP e IGP:

1. BGP seleziona candidati in base a policy e attributi;
2. a parità di criteri principali, il costo IGP verso il next hop può decidere l'uscita.

8.5 eBGP e iBGP

- **eBGP**: sessione tra router di AS diversi;
- **iBGP**: sessione tra router dello stesso AS per distribuire le rotte BGP internamente.

BGP usa TCP porta 179. I vicini sono configurati esplicitamente: non esiste scoperta automatica in broadcast come negli IGP.

Perché TCP?

- consegna affidabile e ordinata degli aggiornamenti;
- controllo di flusso;
- evita di reimplementare ritrasmissioni nel protocollo BGP.

BGP mantiene tuttavia la propria logica di keepalive, hold timer e stato della sessione.

8.6 Finite State Machine

Stati principali:

1. **Idle**: inizializzazione e attesa;
2. **Connect**: tentativo di apertura TCP;
3. **Active**: nuovo tentativo o attesa della connessione;
4. **OpenSent**: messaggio OPEN inviato;
5. **OpenConfirm**: OPEN accettato, attesa KEEPALIVE;
6. **Established**: scambio di UPDATE, KEEPALIVE e NOTIFICATION.

Messaggi BGP:

Messaggio	Funzione
OPEN	ASN, Router ID, Hold Time e capability
UPDATE	Annuncio e ritiro di prefissi con attributi
KEEPALIVE	Mantiene viva la sessione
NOTIFICATION	Segnala errore e chiude la sessione
ROUTE-REFRESH	Richiede riannuncio delle rotte dopo cambio policy

ERRORE COMUNE

Lo stato **Active** non significa che la sessione sia attiva: indica che BGP sta tentando o ritentando di stabilire la connessione TCP.

8.7 NLRI e attributi

Un UPDATE contiene:

- prefissi ritirati;
- attributi del percorso;

- NLRI, cioè i prefissi annunciati.

Gli attributi descrivono come la rotta è stata appresa e come deve essere preferita o propagata.

Classificazione concettuale:

- well-known mandatory;
- well-known discretionary;
- optional transitive;
- optional non-transitive.

8.8 Attributi principali

AS_PATH

Sequenza degli AS attraversati. Funzioni:

- rilevazione dei loop;
- criterio di selezione;
- traffic engineering tramite **AS-path prepending**.

Se un router vede il proprio ASN nell'AS_PATH, rifiuta la rotta.

NEXT_HOP

Indirizzo del router a cui inviare il traffico. Deve essere raggiungibile tramite la tabella di routing. La risoluzione del next hop può essere ricorsiva: BGP indica un indirizzo e l'IGP deve fornire la rotta per raggiungerlo.

LOCAL_PREF

Preferenza interna all'AS. Valore più alto preferito. È distribuito tramite iBGP e non viene inviato agli AS esterni. Il valore predefinito comune è 100.

Usi:

- preferire un customer;
- scegliere un'uscita primaria;
- influenzare il traffico outbound dell'intero AS.

MED

Multi-Exit Discriminator suggerisce a un AS vicino quale collegamento preferire per entrare. Valore più basso preferito. Per impostazione comune viene confrontato tra rotte ricevute dallo stesso AS vicino; il comportamento esatto dipende dalla configurazione.

ORIGIN

Indica l'origine storica della rotta:

- **i**: IGP;
- **e**: EGP;
- **?**: incomplete.

Nel confronto viene normalmente preferito **i**, poi **e**, poi **?**.

COMMUNITY

Etichette applicate alle rotte per attivare policy. Permettono di evitare configurazioni basate solo su prefissi. Esistono community standard, extended e large.

Esempi concettuali:

- “non annunciare a nessuno”;
- “non annunciare a un certo peer”;
- “imposta local preference alta”.

Weight

Attributo locale tipico di Cisco e supportato da alcune implementazioni. Non viene propagato. Un valore maggiore può rendere una rotta preferita solo su quel router.

8.9 Processo di selezione del best path

L'ordine esatto varia tra implementazioni e configurazioni. Una sequenza didattica utile è:

1. route valida con next hop raggiungibile;
2. preferenza locale/weight più alta, se supportata;
3. rotta originata localmente;
4. LOCAL_PREF più alto;
5. AS_PATH più corto;
6. ORIGIN migliore;
7. MED più basso, nei casi confrontabili;
8. eBGP preferito a iBGP;
9. costo IGP più basso verso il NEXT_HOP, cioè hot potato;
10. ulteriori tie-breaker deterministici.

ATTENZIONE

Non memorizzare l'ordine come una verità universale senza citare l'implementazione. All'orale è più importante spiegare il ruolo degli attributi e il fatto che le policy precedono spesso la pura distanza.

8.10 iBGP e problema del full mesh

Una rotta appresa via iBGP non viene, per regola classica, riannunciata a un altro peer iBGP. Questo evita loop perché AS_PATH non cambia dentro lo stesso AS, ma richiede un full mesh tra i router iBGP.

Con n router servirebbero:

[]

sessioni.

Per scalare si usano:

- **Route Reflector (RR)**: riflette le rotte tra client iBGP;
- **Confederation**: divide internamente un AS in sotto-AS.

Il route reflector introduce attributi e regole per evitare loop, come `ORIGINATOR_ID` e `CLUSTER_LIST`.

8.11 Next-hop-self

In iBGP il NEXT_HOP appreso da eBGP può rimanere invariato. Un router interno potrebbe ricevere la rotta ma non conoscere l'indirizzo esterno del next hop.

Soluzione:

```
router bgp 65000
neighbor 10.0.0.2 next-hop-self
```

Il router si presenta come next hop, rendendo la rotta risolvibile tramite l'IGP interno.

8.12 Filtri: prefix-list, AS-path list e route-map

Prefix-list

```
ip prefix-list CUSTOMER-IN seq 5 permit 203.0.113.0/24
ip prefix-list CUSTOMER-IN seq 100 deny 0.0.0.0/0 le 32
```

Applicazione:

```
router bgp 65000
neighbor 192.0.2.2 prefix-list CUSTOMER-IN in
```

Le righe sono valutate in ordine. Se nessuna corrisponde, vale un **deny implicito**.

Operatori:

- **ge**: lunghezza minima;
- **le**: lunghezza massima.

Esempio:

```
permit 10.0.0.0/8 ge 16 le 24
```

accetta sottoprefissi di **10.0.0.0/8** con lunghezza da **/16** a **/24**.

AS-path access-list

Filtra in base al percorso usando espressioni regolari.

```
bgp as-path access-list 10 permit ^65010$
```

Il significato degli underscore e delle ancore dipende dalla sintassi BGP dell'implementazione.

Route-map

Le route-map combinano match e set:

```
route-map BACKUP-IN permit 10
match ip address prefix-list DEFAULT
set local-preference 90
```

Applicazione:

```
router bgp 65000
neighbor 192.0.2.2 route-map BACKUP-IN in
```

Possono modificare LOCAL_PREF, MED, community, next hop, AS path e altri attributi.

8.13 Configurazione FRR: eBGP semplice

AS 65001:

```
configure terminal
router bgp 65001
bgp router-id 1.1.1.1
neighbor 193.10.11.2 remote-as 65002
network 195.11.14.0/24
end
write memory
```

AS 65002:

```
configure terminal
router bgp 65002
bgp router-id 2.2.2.2
neighbor 193.10.11.1 remote-as 65001
end
write memory
```

Verifica:

```
show ip bgp summary
show ip bgp neighbors
show ip bgp
show ip bgp 195.11.14.0/24
show ip route bgp
```

Il prefisso del comando `network` deve normalmente esistere nella RIB locale con la stessa maschera. In alcuni laboratori si disabilita il controllo con:

```
no bgp network import-check
```

ma è meglio comprendere e creare la rotta locale corretta.

A seconda del profilo/versione di FRR, l'eBGP può richiedere una policy esplicita di import/export. Nei laboratori si può incontrare:

```
no bgp ebgp-requires-policy
```

Questo semplifica l'esercizio, ma in produzione la policy esplicita è una protezione importante.

OSSERVAZIONE IN KATHARÁ

Una sessione `Established` non implica che esistano rotte scambiate. Controllare separatamente stato TCP/BGP, policy, tabella BGP, validità del next hop e installazione nella RIB.

8.14 Default route per una stub AS

Un cliente con un solo provider non necessita della full Internet table. Il provider può inviare solo `0.0.0.0/0`:

```
router bgp 65000
neighbor 192.0.2.2 default-originate
```

Il cliente usa la default route per tutte le destinazioni esterne e annuncia al provider solo i propri prefissi.

Vantaggi:

- meno memoria;
- convergenza semplice;
- minore complessità di policy.

8.15 Multi-homing attivo/passivo

Obiettivo: link 1 primario, link 2 backup.

Traffico outbound

Il cliente può impostare `LOCAL_PREF` più bassa sulle rotte ricevute dal link di backup:

```
route-map BACKUP-IN permit 10
set local-preference 90
```

Le rotte del link principale restano, per esempio, a 100.

Traffico inbound

È più difficile, perché decide la rete remota. Possibili strumenti:

- MED, soprattutto tra gli stessi AS;
- AS-path prepending;
- community concordate con il provider;
- annunci più specifici;

- servizi di traffic engineering del provider.

Il MED non è una garanzia globale e non viene confrontato in modo uniforme tra provider diversi.

8.16 Load sharing e prefissi più specifici

Supponiamo di possedere `200.1.0.0/16` e due provider.

Strategia:

- annunciare `/16` a entrambi come copertura;
- annunciare `200.1.0.0/17` preferenzialmente a ISP1;
- annunciare `200.1.128.0/17` preferenzialmente a ISP2.

Per LPM, i `/17` attirano il traffico. Se un link cade e il più specifico scompare, il `/16` dell'altro provider mantiene la raggiungibilità.

ATTENZIONE

Annunciare più specifici aumenta la tabella globale e può essere filtrato se il prefisso è troppo lungo. Il traffic engineering BGP deve rispettare le policy dei provider e le pratiche di aggregazione.

8.17 Sicurezza BGP

Problemi principali:

- route leak;
- prefix hijack;
- annunci accidentali di default o full table;
- AS path non autorizzati;
- sessioni non protette.

Contromisure:

- filtri di prefisso e max-prefix;
- filtri AS_PATH;
- autenticazione TCP, dove usata;
- Generalized TTL Security Mechanism;
- RPKI e Route Origin Validation;
- community di blackholing controllato;
- monitoraggio degli annunci.

RPKI verifica che un AS sia autorizzato a originare un prefisso tramite ROA. Non valida l'intero percorso AS_PATH, ma riduce gli hijack di origine.

8.18 OSPF vs BGP

Aspetto	OSPF	BGP
Ambito	Intra-AS	Inter-AS e grandi fabric
Modello	Link State	Path Vector
Obiettivo	Minimo costo tecnico	Policy e raggiungibilità
Scoperta vicini	Hello su link	Configurazione esplicita
Trasporto	IP protocol 89	TCP 179
Informazione	Topologia dell'area	Prefissi + attributi
Loop prevention	LSDB/SPF e sequenze	AS_PATH, regole iBGP
Convergenza	Rapida in domini controllati	Più conservativa, dipende da policy
Scala	AS/enterprise	Internet globale

8.19 Scheda finale: BGP

Voce	Sintesi
Scopo	Scambio di raggiungibilità e policy tra AS; anche underlay di grandi data center
Algoritmo	Path Vector con processo decisionale basato su attributi
Metrica	Non una singola metrica; LOCAL_PREF, AS_PATH, MED, costo IGP e policy
Vantaggi	Scalabilità globale, policy ricche, aggregazione, controllo degli annunci
Svantaggi	Complessità, convergenza conservativa, rischio di leak/hijack, forte dipendenza dalle policy
Trasporto	TCP 179
Casi d'uso	ISP, multi-homing, Internet, data center Clos
Domande d'orale	eBGP vs iBGP; best path; LOCAL_PREF vs MED; hot potato; no-free-transit; next-hop-self; route reflector; prefix-list/route-map; default route

9. BGP nei data center, Clos, Fat-Tree ed ECMP

9.1 Perché le topologie tradizionali non bastano

Le applicazioni cloud generano molto traffico **east-west**, cioè tra server. Una gerarchia rigida con pochi collegamenti verso il core crea colli di bottiglia e domini di guasto ampi.

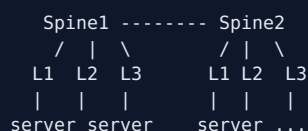
Le reti **Clos** e **Fat-Tree** costruiscono una fabric con molti percorsi paralleli di uguale costo.

Strati tipici:

- **Leaf**: collega server e appliance;
- **Spine**: collega tutti i leaf;
- **Super-spine** o livelli superiori: collegano più pod o fabric.

Il termine PoD può indicare un blocco ripetibile della fabric; la nomenclatura esatta dipende dall'architettura.

Schema leaf-spine:



Ogni leaf è collegato a ogni spine. Due server su leaf diversi dispongono di più cammini della stessa lunghezza.

9.2 ECMP

Equal-Cost Multi-Path installa più next hop con costo equivalente. Il forwarding distribuisce i flussi tramite hashing su campi come:

- IP sorgente;
- IP destinazione;
- protocollo;
- porta sorgente;
- porta destinazione.

L'hash per flusso evita che pacchetti della stessa connessione prendano percorsi diversi e arrivino fuori ordine.

Possibili problemi:

- flussi "elefante" e "topo": pochi flussi grandi possono sbilanciare l'hash;
- guasto di un next hop: i flussi vengono rimappati;
- polarizzazione dell'hash;
- visibilità incompleta con traceroute tradizionale.

9.3 Perché usare BGP come underlay

In grandi fabric si usa spesso eBGP tra apparati adiacenti:

- policy e comportamento ben conosciuti;
- isolamento dei domini di errore;
- nessun flooding globale della topologia;
- configurazione uniforme;
- supporto a multipath;
- scalabilità tramite ASN distinti.

Questo non significa che OSPF sia “sbagliato”: è una scelta architetturale. BGP scambia raggiungibilità, mentre la topologia Clos è regolare e progettata per molti percorsi equivalenti.

9.4 ASN privati

Intervalli privati:

- 64512-65534 per ASN a 16 bit;
- 4200000000-4294967294 per ASN a 32 bit.

Una convenzione possibile:

- ASN distinto per ogni leaf;
- ASN condiviso o distinto per spine, secondo il design;
- gruppi coerenti per pod o livello.

L’obiettivo è evitare loop e semplificare le policy, non replicare necessariamente la struttura amministrativa di Internet.

9.5 BGP multipath

Per installare più percorsi BGP sono necessari comandi dipendenti dall’implementazione, per esempio:

```
router bgp 65001
maximum-paths 8
bgp bestpath as-path multipath-relax
```

`multipath-relax` permette di considerare equivalenti percorsi con AS_PATH della stessa lunghezza anche se la sequenza di ASN non è identica. Va usato nel contesto di una topologia controllata.

Verifica:

```
show ip bgp 10.20.0.0/16
show ip route 10.20.0.0/16
```

La tabella BGP può mostrare più candidati, mentre la RIB deve confermare quanti next hop siano stati installati.

9.6 Convergenza rapida

Nei data center i tempi predefiniti di BGP possono essere troppo lenti. Possibili ottimizzazioni:

- keepalive/hold timer ridotti;
- connect retry più breve;
- advertisement interval ridotto;
- BFD;
- fast external fallover;
- rilevazione del link fisico;
- graceful restart, con cautela.

Esempio didattico:

```
neighbor 10.0.0.2 timers 3 9
```

Timer aggressivi aumentano il carico e possono causare instabilità in presenza di jitter o CPU sovraccarica. BFD è spesso preferibile per separare la rilevazione rapida dal protocollo BGP.

9.7 Traceroute e percorsi multipli

Il traceroute tradizionale può cambiare campi di trasporto tra una sonda e l’altra. Un router ECMP può quindi assegnare le sonde a next hop diversi, facendo apparire percorsi impossibili o mescolati.

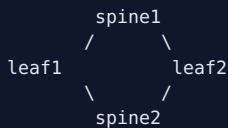
Paris Traceroute mantiene costanti i campi usati dall'hash e ricostruisce meglio un singolo percorso. Strumenti come Dublin Traceroute cercano di esplorare più rami della topologia.

DOMANDA TIPICA D'ESAME

“Perché traceroute può essere ingannevole in presenza di ECMP?” Perché le sonde possono avere tuple diverse e venire hashate su percorsi differenti; le righe non appartengono necessariamente a un unico cammino coerente.

9.8 Laboratorio concettuale

Topologia:



Obiettivi:

1. creare peering eBGP su ogni link;
2. annunciare le loopback dei leaf;
3. abilitare multipath;
4. verificare più next hop;
5. generare più flussi con `iperf3 -P 8`;
6. interrompere un link e misurare il tempo di ripristino;
7. usare tcpdump sulla porta 179 per osservare NOTIFICATION/UPDATE e ristabilimento.

9.9 Tabella riassuntiva

Concetto	Funzione	Punto critico
Clos/Fat-Tree	Molti percorsi paralleli	Cablaggio e controllo della fabric
Leaf	Accesso server	Aggrega traffico degli host
Spine	Interconnessione leaf	Ogni leaf deve raggiungere ogni spine
ECMP	Usa più percorsi equivalenti	Hash per flusso e possibili sbilanciamenti
BGP underlay	Distribuisce prefissi della fabric	Policy e timer coerenti
Multipath relax	Allenta uguaglianza AS_PATH	Solo in design controllati
BFD	Rilevazione rapida guasti	Configurare timer sostenibili
Paris Traceroute	Evita artefatti ECMP	Non scopre automaticamente ogni ramo

10. Livello di trasporto e trasferimento affidabile

10.1 Perché serve il livello di trasporto

IP consegna datagrammi da un host a un altro, ma un computer esegue molti processi contemporaneamente. Il livello di trasporto fornisce una comunicazione **processo-processo** usando i numeri di porta.

Le funzioni principali sono:

- multiplexing e demultiplexing;
- segmentazione e ricomposizione;
- affidabilità, quando richiesta;
- controllo di flusso;
- controllo della congestione;
- gestione della connessione, nel caso di TCP.

10.2 Multiplexing e demultiplexing

Multiplexing

In uscita, il livello di trasporto riceve dati da più socket, aggiunge l'header TCP o UDP e consegna i segmenti a IP.

Demultiplexing

In ingresso, legge indirizzi, protocollo e porte per scegliere il socket destinatario.

Una semplificazione didattica utile:

- socket UDP non connesso: identificato principalmente da IP locale e porta locale;
- connessione TCP: identificata dalla quadrupla:

(IP sorgente, porta sorgente, IP destinazione, porta destinazione)

In pratica anche UDP può essere "connesso" a un peer e il kernel usa regole più articolate, ma la distinzione chiarisce perché un server TCP può gestire molte connessioni sulla stessa porta.

Esempio:

```
Client A 10.0.0.1:50100 ---> Server 10.0.0.10:443
Client B 10.0.0.2:50200 ---> Server 10.0.0.10:443
```

Le due connessioni condividono la porta 443 del server ma hanno tuple diverse.

10.3 UDP

UDP fornisce:

- multiplexing tramite porte;
- checksum;
- consegna best-effort senza connessione;
- nessuna garanzia di ordine, consegna o non duplicazione;
- nessun controllo di congestione incorporato nel protocollo.

Header UDP:

Campo	Dimensione
Source Port	16 bit

Campo	Dimensione
Destination Port	16 bit
Length	16 bit
Checksum	16 bit

Il checksum UDP è opzionale in IPv4 ma obbligatorio in IPv6, salvo eccezioni molto specifiche.

Casi d'uso:

- DNS;
- VoIP;
- streaming interattivo;
- gaming;
- QUIC;
- telemetria;
- protocolli di routing come RIP.

ATTENZIONE

“UDP è più veloce di TCP” è una semplificazione. UDP ha meno meccanismi, ma prestazioni e affidabilità dipendono dall'applicazione. QUIC, per esempio, usa UDP ma implementa controllo di congestione, affidabilità e cifratura nello user space.

10.4 TCP

TCP offre un flusso di byte:

- orientato alla connessione;
- affidabile;
- ordinato;
- full-duplex;
- con controllo di flusso;
- con controllo della congestione.

Non conserva i confini dei messaggi applicativi. Due `send()` possono essere letti con una sola `recv()` o viceversa: l'applicazione deve definire il proprio framing.

10.5 Costruire teoricamente un protocollo affidabile

Il modello **Reliable Data Transfer (RDT)** mostra come si aggiungono progressivamente funzioni per affrontare gli errori del canale.

Errore nei bit: checksum

Il mittente calcola un valore di controllo. Il ricevitore lo ricalcola:

- uguale: il segmento è presumibilmente integro;
- diverso: è stato rilevato un errore.

Un checksum non garantisce la rilevazione di ogni possibile errore, ma riduce fortemente la probabilità di corruzioni non rilevate.

Feedback: ACK e NAK

- **ACK**: dato ricevuto correttamente;
- **NAK/NAACK**: dato corrotto o non accettato.

Il mittente ritrasmette dopo un NAK.

ACK corrotto e duplicati: numeri di sequenza

Se l'ACK è corrotto, il mittente può ritrasmettere. Il ricevitore deve riconoscere il duplicato tramite un **sequence number** e non consegnare due volte gli stessi dati.

Nei protocolli stop-and-wait basta alternare i numeri 0 e 1, perché può esserci un solo pacchetto non confermato.

Perdita totale: timer e timeout

Se dato o ACK scompaiono, senza timer la comunicazione si blocca. Il mittente:

1. invia il pacchetto;
2. avvia un timer;
3. riceve ACK e annulla il timer, oppure;
4. al timeout ritrasmette.

Un timeout troppo corto causa ritrasmissioni spurie; troppo lungo rallenta il recupero.

DOMANDA TIPICA D'ESAME

“Perché servono i numeri di sequenza se esistono già ACK e timeout?” Per distinguere dati nuovi da ritrasmissioni e impedire la consegna duplicata quando un ACK si perde o è corrotto.

10.6 Stop-and-Wait e utilizzazione

Con Stop-and-Wait il mittente invia un pacchetto e attende l'ACK prima del successivo.

Utilizzazione approssimata del mittente:

[$U =$]

con:

- L : lunghezza del pacchetto in bit;
- R : bitrate;
- L/R : tempo di trasmissione.

Su un link veloce con RTT elevato, L/R è molto piccolo rispetto a RTT: la capacità resta inutilizzata.

Esempio: pacchetto 12.000 bit, link 1 Gbit/s, RTT 40 ms:

[$L/R = 12;s$]

L'utilizzazione è circa 0,03%. Serve inviare più dati senza attendere ogni ACK.

10.7 Pipelining e sliding window

Il mittente mantiene una finestra di pacchetti inviabili. La finestra “scorre” quando arrivano ACK.

Benefici:

- più dati in volo;
- migliore utilizzazione del BDP;
- minore inattività.

Costi:

- più buffer;
- numeri di sequenza più ampi;
- logica di ritrasmissione più complessa.

10.8 Go-Back-N

Caratteristiche:

- finestra del mittente di dimensione N ;

- ricevitore semplice, tipicamente accetta solo il prossimo pacchetto atteso;
- ACK cumulativi;
- un timer, normalmente per il pacchetto non confermato più vecchio.

Esempio:

```
Ricevuti: 1, 2, [3 perso], 4, 5
Ricevitore: scarta 4 e 5, continua a inviare ACK 2
Timeout: il mittente ritrasmette 3, 4, 5
```

Vantaggio: ricevitore semplice.

Svantaggio: spreca banda quando la finestra è grande e la perdita è isolata.

10.9 Selective Repeat

Caratteristiche:

- ACK individuali;
- buffer dei pacchetti fuori ordine;
- timer per pacchetto o meccanismi equivalenti;
- ritrasmissione selettiva.

Esempio:

```
Ricevuti: 1, 2, [3 perso], 4, 5
Ricevitore: memorizza 4 e 5, invia ACK 4 e ACK 5
Mittente: ritrasmette solo 3
```

La finestra e lo spazio dei numeri di sequenza devono essere dimensionati in modo da evitare ambiguità tra pacchetti vecchi e nuovi. Con numerazione modulo 2^k , una regola classica richiede finestra non superiore a metà dello spazio dei numeri.

10.10 Go-Back-N vs Selective Repeat

Aspetto	Go-Back-N	Selective Repeat
ACK	Cumulativi	Individuali/selettivi
Fuori ordine	Scartati	Bufferizzati
Ritrasmissione	Dal pacchetto perso in poi	Solo pacchetti mancanti
Complessità ricevitore	Bassa	Alta
Efficienza con perdita	Peggior	Migliore
Memoria	Minore	Maggiore

10.11 TCP come protocollo affidabile reale

TCP combina idee di entrambi i modelli:

- numeri di sequenza per **byte**, non per segmento;
- ACK cumulativi che indicano il prossimo byte atteso;
- ritrasmissione su timeout;
- fast retransmit su ACK duplicati;
- buffer di dati fuori ordine in molte implementazioni;
- SACK opzionale per segnalare blocchi ricevuti.

Campi importanti dell'header:

Campo	Funzione
Source/Destination Port	Demultiplexing

Campo	Funzione
Sequence Number	Posizione del primo byte nel flusso
Acknowledgment Number	Prossimo byte atteso
Flags	SYN, ACK, FIN, RST, PSH, ECE, CWR
Window	Receive window per flow control
Checksum	Integrità
Options	MSS, Window Scale, SACK, Timestamp

10.12 Three-Way Handshake

```

Client                               Server
| ----- SYN, seq=x -----> |
| <----- SYN+ACK, seq=y, ack=x+1 ----- |
| ----- ACK, ack=y+1 -----> |

```

Scopi:

- sincronizzare i numeri di sequenza;
- confermare raggiungibilità bidirezionale;
- negoziare opzioni;
- creare stato alle due estremità.

Il server può trovarsi in `LISTEN`, poi `SYN-RECEIVED`; il client passa da `SYN-SENT` a `ESTABLISHED`.

Chiusura

La chiusura ordinata usa FIN e ACK in ciascuna direzione. Essendo full-duplex, le due metà possono chiudersi separatamente. RST interrompe bruscamente la connessione.

10.13 Flow control

Il ricevitore pubblica la **receive window (rwnd)**: spazio disponibile nel buffer. Il mittente limita i dati non confermati per non sommergere il ricevitore.

Il limite effettivo di invio è approssimativamente:

[FlightSize (rwnd, cwnd)]

- `rwnd` : capacità del ricevitore;
- `cwnd` : stima della capacità della rete.

ERRORE COMUNE

Flow control e congestion control non sono sinonimi. Il primo protegge il ricevitore; il secondo protegge la rete. TCP applica entrambi contemporaneamente.

10.14 RTT e Retransmission Timeout

TCP stima RTT con una media smussata:

[EstimatedRTT = (1-α)EstimatedRTT + SampleRTT]

Stima anche la variabilità e calcola un RTO sufficientemente superiore al RTT. I dettagli moderni sono definiti dagli standard e includono backoff esponenziale.

Un timeout deve adattarsi alla rete: un valore fisso sarebbe troppo corto su percorsi lenti e troppo lungo su LAN veloci.

10.15 Schede finali: UDP e TCP

UDP

Voce	Sintesi
Scopo	Trasporto minimale orientato ai datagrammi
Vantaggi	Header piccolo, nessun handshake, controllo applicativo
Svantaggi	Nessuna affidabilità/ordine/congestion control integrati
Algoritmo	Best effort con checksum e porte
Metrica	Non applicabile
Casi d'uso	DNS, VoIP, gaming, QUIC, telemetria
Domande d'orale	Header; checksum; demultiplexing; perché QUIC usa UDP

TCP

Voce	Sintesi
Scopo	Flusso di byte affidabile e ordinato
Vantaggi	Affidabilità, controllo di flusso e congestione, ampia compatibilità
Svantaggi	Stato, handshake, head-of-line a livello del flusso, evoluzione kernel lenta
Algoritmo	ACK, SEQ, timer, sliding window, congestion control
Metrica	cwnd/rwnd, RTT, RTO; non una metrica di routing
Casi d'uso	Web, SSH, file transfer, database
Domande d'orale	4-tuple; handshake; ACK cumulativi; flow vs congestion control; timeout e fast retransmit

11. Controllo della congestione: Reno, CUBIC e QUIC

11.1 Congestione e collasso

La congestione si verifica quando il traffico offerto supera, per un periodo significativo, la capacità di link, code o nodi.

Effetti:

- crescita delle code;
- aumento del ritardo;
- perdita di pacchetti;
- ritrasmissioni;
- consumo di capacità per dati destinati a essere scartati;
- possibile collasso del throughput utile.

Il controllo della congestione deve trovare un punto operativo efficiente senza penalizzare ingiustamente altri flussi.

Segnali di congestione

- perdita rilevata da timeout;
- ACK duplicati;
- marcatura ECN;
- aumento del RTT e del tempo in coda;
- segnali espliciti della rete, in protocolli specializzati.

11.2 Congestion Window e self-clocking

TCP mantiene una **congestion window (cwnd)** che limita i byte non confermati.

Gli ACK fungono da clock:

- un ACK indica che dati hanno lasciato il percorso e capacità si è liberata;
- il flusso degli ACK regola il ritmo di invio;
- perdita o marcatura suggeriscono eccesso di carico.

Questo meccanismo è distribuito: ogni mittente adatta la propria velocità osservando il percorso.

11.3 Slow Start

All'inizio TCP non conosce la capacità. Parte da una **Initial Window**, oggi spesso maggiore di 1 MSS, e aumenta rapidamente.

Regola concettuale:

- per ogni ACK, `cwnd` aumenta circa di un MSS;
- in un RTT vengono confermati circa `cwnd` byte;
- la finestra raddoppia approssimativamente ogni RTT.

Il nome "slow" è storico: la crescita è esponenziale ed è aggressiva.

La fase termina quando:

- `cwnd` raggiunge `ssthresh`;
- viene rilevata congestione;
- intervengono limiti di `rwnd` o applicazione.

11.4 Congestion Avoidance

Oltre `ssthresh`, l'aumento diventa additivo:

`[ cwnd cwnd + 1;MSS;per;RTT ]`

Implementazione per ACK:

`[ cwnd ]`

La somma degli incrementi in un RTT produce circa un MSS.

11.5 Timeout e ACK duplicati

Timeout

Indica che la rete non ha fornito feedback in tempo. La reazione classica:

- `ssthresh` circa metà della flight size;
- `cwnd` torna alla finestra iniziale;
- riparte Slow Start;
- RTO subisce backoff.

Tre ACK duplicati

Se arrivano ACK con lo stesso acknowledgment number, segmenti successivi sono probabilmente arrivati ma manca un "buco". Dopo tre duplicate ACK:

- **Fast Retransmit**: ritrasmette senza attendere il timeout;
- **Fast Recovery** in Reno: riduce la finestra ma evita di ripartire completamente.

Tahoe, invece, dopo la perdita torna a Slow Start anche in caso di triple duplicate ACK.

DOMANDA TIPICA D'ESAME

"Perché tre ACK duplicati sono considerati un segnale meno grave del timeout?" Perché dimostrano che il percorso continua a consegnare segmenti successivi e quindi esiste ancora capacità; il timeout indica assenza di feedback.

11.6 Reno, Tahoe e AIMD

TCP Reno segue l'idea **AIMD**:

- Additive Increase: aumento lineare durante congestion avoidance;
- Multiplicative Decrease: riduzione proporzionale dopo congestione.

AIMD tende a convergere verso efficienza ed equità tra flussi simili che condividono un collo di bottiglia.

Differenza semplificata:

Evento	Tahoe	Reno
Timeout	Riduzione e Slow Start	Riduzione e Slow Start
3 duplicate ACK	Fast Retransmit, poi Slow Start	Fast Retransmit + Fast Recovery

11.7 Limiti RTT di AIMD

Con aumento di circa un MSS per RTT, un flusso con RTT basso aumenta più spesso la propria finestra rispetto a uno con RTT alto. Di conseguenza, flussi Reno con RTT diversi possono ottenere quote non eque.

Questo **RTT bias** è una conseguenza della dinamica temporale, non di una policy esplicita.

11.8 TCP CUBIC

CUBIC è progettato per reti ad alto Bandwidth-Delay Product. Usa una funzione cubica del tempo trascorso dall'ultima congestione, invece di dipendere direttamente da incrementi per RTT.

La finestra evolve intorno a `Wmax`, valore raggiunto prima dell'ultima perdita:

1. **regione concava**: recupera rapidamente verso `Wmax`;
2. **plateau**: esplora con cautela vicino al punto noto;
3. **regione convessa**: supera `Wmax` e cerca nuova capacità in modo più aggressivo.

Dopo perdita, molte implementazioni CUBIC mantengono una frazione della finestra, tipicamente intorno al 70%, invece del dimezzamento classico.

Vantaggi:

- migliore uso di link veloci e lunghi;
- minore dipendenza dal RTT rispetto a Reno;
- compatibilità con una regione TCP-friendly.

Limiti:

- resta principalmente loss-based;
- può riempire le code prima di osservare perdita;
- interagisce con AQM, ECN e buffer del percorso.

11.9 BBR

BBR adotta un approccio model-based. Stima:

- banda massima di consegna;
- RTT minimo del percorso.

L'obiettivo è operare vicino al BDP senza riempire persistentemente le code. Usa pacing e controllo del volume in volo.

BBR non interpreta la perdita come unico segnale di congestione. Le sue versioni e interazioni di fairness sono oggetto di continua evoluzione; nei laboratori si può confrontare con CUBIC tramite:

```
sysctl net.ipv4.tcp_available_congestion_control  
iperf3 -c 10.0.0.2 -C cubic  
iperf3 -c 10.0.0.2 -C bbr
```

L'algoritmo deve essere disponibile e abilitato nel kernel.

11.10 QUIC

QUIC è un protocollo di trasporto sicuro usato da HTTP/3. Opera sopra UDP ma implementa nello user space:

- connessione;
- affidabilità;
- controllo di congestione;
- multiplexing di stream;
- TLS 1.3 integrato;
- migrazione della connessione.

Perché non modificare TCP?

TCP è implementato nei kernel e attraversa middlebox che assumono comportamenti storici. Innovazioni profonde richiedono tempi lunghi. QUIC usa UDP come substrato e può essere aggiornato insieme alle applicazioni.

11.11 Vantaggi di QUIC

Setup rapido

TCP + TLS richiedono handshake distinti. QUIC integra il handshake crittografico e può ridurre il numero di RTT. Con ripresa di una sessione già conosciuta può inviare dati **0-RTT**, ma tali dati hanno vincoli di sicurezza e rischio di replay; l'applicazione deve usarli solo per operazioni idempotenti o protette.

Stream indipendenti

Una connessione QUIC contiene più stream. La perdita di un pacchetto può bloccare i dati mancanti nello stesso stream, ma non necessariamente gli altri stream.

TCP, essendo un unico flusso ordinato, può causare head-of-line blocking tra richieste multiplexate nello stesso flusso applicativo.

Sicurezza integrata

TLS 1.3 è parte del protocollo. Molti metadati di trasporto sono protetti, riducendo l'interferenza delle middle-box.

Connection migration

QUIC usa Connection ID non legati rigidamente alla 4-tuple. Un dispositivo può passare da Wi-Fi a rete mobile e mantenere la connessione, dopo validazione del nuovo percorso.

Evolubilità

Lo user space e la cifratura permettono aggiornamenti più rapidi e limitano l'ossificazione del protocollo.

11.12 TCP vs QUIC

Aspetto	TCP + TLS	QUIC
Substrato	TCP nel kernel	UDP + logica user space
Sicurezza	TLS sopra TCP	TLS 1.3 integrato
Setup	TCP handshake + TLS	Handshake combinato
0-RTT	TLS resumption, dopo TCP	Supportato in ripresa QUIC
Multiplexing	Applicativo; HOL del flusso TCP	Stream indipendenti
Migrazione IP	Connessione normalmente si rompe	Connection ID e path validation
Aggiornabilità	Dipende dal kernel	Applicazione/libreria
Congestion control	CUBIC, BBR, ecc.	Algoritmi analoghi implementati in QUIC

11.13 Test di congestion control con iperf3 e tc

Scenario:

```
# Server
iperf3 -s

# Client baseline
iperf3 -c 10.0.0.2 -t 30 -C cubic

# Introduci 40 ms di ritardo e 0,5% di perdita
sudo tc qdisc add dev eth0 root netem delay 40ms loss 0.5%

# Ripeti
iperf3 -c 10.0.0.2 -t 30 -C cubic

# Se disponibile
iperf3 -c 10.0.0.2 -t 30 -C bbr
```

Osservare:

- throughput medio;
- ritrasmissioni;
- tempo di crescita;
- RTT del ping;
- stabilità del bitrate.

Per un confronto corretto:

- ripetere più volte;
- usare stessa durata;
- svuotare o stabilizzare le code;
- controllare CPU;
- verificare la direzione;
- non generalizzare da un unico test.

11.14 Scheda finale: QUIC

Voce	Sintesi
Scopo	Trasporto sicuro, evolvibile e multiplexato per applicazioni moderne
Vantaggi	Setup ridotto, TLS 1.3, stream indipendenti, migrazione, user space
Svantaggi	Complessità, CPU, visibilità ridotta per middlebox, dipendenza da UDP consentito
Algoritmo	Affidabilità per stream + congestion control + crittografia
Metrica	RTT, delivery rate, loss/ECN secondo algoritmo
Casi d'uso	HTTP/3, web, servizi mobili
Domande d'orale	Perché sopra UDP; 0-RTT; HOL; Connection ID; TLS integrato; differenze da TCP

12. Fairness, AQM, ECN e isolamento del traffico

12.1 Efficienza ed equità

Una rete può massimizzare il throughput totale assegnando quasi tutta la capacità ai flussi più efficienti, ma questo può affamare altri utenti. Il controllo della congestione deve bilanciare:

- **efficienza**: usare la capacità disponibile;
- **equità**: distribuire risorse secondo un criterio accettabile;
- **stabilità**: evitare oscillazioni;
- **reattività**: adattarsi ai cambiamenti;
- **bassa latenza**: non riempire inutilmente le code.

Non esiste una sola definizione di fairness adatta a ogni rete.

12.2 Max-Min Fairness

Un'allocazione è max-min fair se non è possibile aumentare la velocità di un flusso senza diminuire quella di un flusso con velocità minore o uguale.

Algoritmo di water filling:

1. aumentare allo stesso ritmo tutti i flussi;
2. quando un link satura, bloccare i flussi limitati da quel link;
3. continuare ad aumentare gli altri;
4. ripetere fino a saturare ogni collo di bottiglia.

Ogni flusso deve avere almeno un **bottleneck link** saturo sul quale possiede una quota non inferiore a quella dei flussi concorrenti rilevanti.

Esempio:

```
Link A: 10 Mbit/s condiviso da F1 e F2
Link B: 20 Mbit/s usato solo da F2 dopo A

Max-min: F1 = 5, F2 = 5 Mbit/s
```

Il secondo link resta parzialmente inutilizzato perché F2 è già limitato da A.

12.3 Proportional Fairness

Massimizza:

$\sum_i (x_i)$

Il log attribuisce grande utilità agli aumenti dei flussi piccoli e utilità marginale decrescente ai flussi grandi. Rispetto a max-min, può accettare una distribuzione meno uniforme in cambio di maggiore efficienza totale.

Una variazione percentuale positiva aggregata non deve poter superare la perdita percentuale aggregata rispetto all'allocazione ottima.

È utile in scenari in cui flussi attraversano numeri diversi di colli di bottiglia, come il "parking lot".

12.4 Utility Fairness

Si definisce una funzione di utilità $U_i(x_i)$ per ogni flusso e si massimizza:

$\sum_i U_i(x_i)$

soggetta ai vincoli di capacità.

Scelte diverse di **U** producono criteri diversi. Le funzioni concave favoriscono stabilità e rendimenti decrescenti. Il framework permette di rappresentare preferenze, priorità e classi di servizio.

12.5 AIMD e convergenza

AIMD combina:

- aumento additivo in assenza di congestione;
- diminuzione moltiplicativa in presenza di congestione.

Considerando due flussi che condividono una capacità **C**, il piano delle velocità ha:

- linea di efficienza $x_1 + x_2 = C$;
- linea di equità $x_1 = x_2$.

L'aumento additivo muove parallelamente alla linea di equità; la diminuzione moltiplicativa avvicina all'origine mantenendo il rapporto. La sequenza tende verso l'intersezione tra efficienza ed equità.

Per la convergenza classica:

- l'aumento deve avere componente additiva positiva;
- la diminuzione deve essere moltiplicativa con fattore tra 0 e 1.

12.6 Buffer, tail drop e bufferbloat

Una coda **tail drop** accetta pacchetti finché il buffer è pieno e poi scarta i nuovi arrivi.

Problemi:

- ritardo elevato se il buffer resta pieno;
- perdite a raffica;
- sincronizzazione globale dei flussi TCP;
- bufferbloat.

Il **bufferbloat** è l'aumento persistente della latenza dovuto a code eccessive. Un link può mostrare throughput elevato ma esperienza interattiva pessima.

Test:

```
# Terminale 1: carico
iperf3 -c 10.0.0.2 -t 60

# Terminale 2: RTT durante il carico
ping 10.0.0.2
```

Se l'RTT cresce di centinaia di millisecondi sotto saturazione, è probabile una coda eccessiva.

12.7 Active Queue Management

L'**AQM** segnala congestione prima che il buffer sia completamente pieno.

RED

Random Early Detection calcola una media smussata della lunghezza della coda:

- sotto una soglia minima: non scarta;
- tra soglia minima e massima: scarta o marca con probabilità crescente;
- sopra la soglia massima: scarta/segna con probabilità elevata.

Obiettivi:

- segnalare congestione precocemente;
- evitare sincronizzazione;
- mantenere la coda media controllata.

Limiti:

- parametri difficili da scegliere;
- sensibilità al carico;
- prestazioni non sempre robuste.

CoDel e fq_codel

CoDel usa il tempo di permanenza in coda invece della sola lunghezza. `fq_codel` aggiunge fair queuing per separare i flussi.

Vantaggi:

- meno tuning manuale;
- bassa latenza;
- isolamento tra flussi;
- protezione dei flussi piccoli dai flussi grandi.

12.8 ECN

Explicit Congestion Notification permette di segnalare congestione senza scartare il pacchetto.

Meccanismo semplificato:

1. gli endpoint dichiarano supporto ECN;
2. il router AQM marca il campo ECN nell'header IP con `CE` invece di scartare;
3. il ricevitore segnala la marcatura al mittente con flag TCP `ECE`;
4. il mittente riduce la finestra e usa `CWR` per indicare la reazione.

Vantaggi:

- evita perdita e ritrasmissione;
- segnala prima della saturazione;
- utile in data center e reti a bassa latenza.

ECN richiede supporto coerente di endpoint e rete. Se un apparato cancella o gestisce male le marcature, i benefici si riducono.

12.9 DCTCP

Data Center TCP usa ECN per mantenere code molto corte.

Invece di reagire in modo binario, stima la frazione di byte marcati e aggiorna un parametro `alpha`. La riduzione della finestra è proporzionale alla congestione osservata.

Risultato:

- throughput elevato;
- code corte;
- reazione graduata;
- minore latenza per flussi brevi.

DCTCP presuppone un ambiente controllato con soglie ECN adeguate. Non è semplicemente "TCP con ECN": modifica l'algoritmo di reazione.

12.10 Applicazioni TCP-friendly

Un'applicazione UDP non deve ignorare la congestione. Se invia sempre al massimo, può sottrarre risorse ai flussi adattivi.

Approcci:

- controllo di congestione applicativo;
- rate adaptation;
- TFRC (TCP-Friendly Rate Control);

- algoritmi di QUIC;
- feedback del ricevitore;
- bitrate adattivo per video.

La formula loss-throughput collega velocità sostenibile, RTT e probabilità di perdita. TFRC cerca un andamento più regolare di TCP, utile per media continui.

12.11 Class-Based Queuing e schedulazione

In reti gestite si isolano classi di traffico:

- voce;
- video;
- traffico interattivo;
- backup;
- best effort.

Meccanismi:

- priority queuing;
- Weighted Round Robin;
- Deficit Round Robin;
- Weighted Fair Queuing;
- HTB;
- shaping e policing.

Una coda prioritaria assoluta può affamare le altre classi; va limitata o controllata.

12.12 Fairness non significa sempre uguaglianza

Esempi:

- due utenti best-effort: quota simile può essere appropriata;
- VoIP e backup: la voce necessita poca banda ma bassa latenza;
- cliente premium: la policy può assegnare più capacità;
- flusso con RTT alto: uguaglianza per connessione può non coincidere con uguaglianza per utente;
- molte connessioni parallele: un'app può ottenere più banda aprendo più flussi.

La fairness va definita rispetto all'unità: flusso, utente, applicazione, classe o organizzazione.

DOMANDA TIPICA D'ESAME

“Perché massimizzare il throughput non garantisce fairness?” Perché un'allocazione Pareto-efficiente può assegnare quasi tutto a pochi flussi. Servono funzioni di utilità o criteri come max-min/proportional fairness.

12.13 Tabella comparativa

Meccanismo	Segnale	Reazione/obiettivo	Ambiente tipico
Reno	Loss/dup ACK	AIMD	Internet classica
CUBIC	Loss + tempo	Crescita cubica	Linux, reti ad alto BDP
BBR	Delivery rate + min RTT	Modello di banda/RTT	WAN moderne
RED	Lunghezza media coda	Drop/mark probabilistico	AQM classica
CoDel	Sojourn time	Controllo del ritardo	Access network
fq_codel	Sojourn + fair queues	Bassa latenza e isolamento	Host/router Linux
ECN	Marcatura	Riduzione senza perdita	Reti compatibili

Meccanismo	Segnale	Reazione/obiettivo	Ambiente tipico
DCTCP	Frazione ECN	Riduzione proporzionale	Data center
HTB/WFQ	Classi/pesi	Garanzie e isolamento	Reti amministrate

13. Collegamenti trasversali tra i capitoli

13.1 Dalla risoluzione del vicino al routing globale

Un singolo pacchetto diretto a Internet attraversa più meccanismi:

1. l'applicazione genera dati;
2. TCP/UDP/QUIC aggiunge porte e funzioni di trasporto;
3. IP seleziona la rotta con LPM;
4. ARP o NDP risolve il MAC del next hop locale;
5. il frame attraversa il link;
6. ogni router ripete forwarding e risoluzione sul proprio link;
7. OSPF/IS-IS può aver calcolato i percorsi interni;
8. BGP può aver scelto il punto di uscita o il percorso inter-AS.

COLLEGAMENTO CHIAVE

ARP/NDP non “instradano” e OSPF/BGP non consegnano frame Ethernet. Ogni protocollo risolve un problema a un livello e usa i risultati degli altri.

13.2 OSPF e BGP

In un AS di grandi dimensioni:

- BGP apprende un prefisso esterno e indica un NEXT_HOP;
- l'IGP, spesso OSPF, rende raggiungibile quel NEXT_HOP;
- il costo OSPF può decidere il punto di uscita a parità di policy BGP;
- `next-hop-self` modifica l'interazione tra i due.

Se l'IGP perde la rotta al next hop, la rotta BGP può restare nel database ma diventare non valida per il forwarding.

13.3 OSPF, BFD e convergenza

OSPF possiede timer Hello/Dead; BGP possiede Keepalive/Hold. BFD può fornire a entrambi una rilevazione rapida e uniforme del guasto. La convergenza finale dipende comunque da propagazione, calcolo e installazione delle rotte.

13.4 ECMP e trasporto

ECMP distribuisce flussi con hashing. Il numero di connessioni influenza la distribuzione:

- un singolo flusso TCP usa normalmente un solo percorso;
- `iperf3 -P 8` crea più flussi e può usare più next hop;
- QUIC può contenere molti stream ma resta normalmente una singola connessione/tupla ai fini dell'hash, salvo migrazioni o politiche diverse.

Questo spiega perché più flussi paralleli possono ottenere throughput aggregato maggiore in una fabric.

13.5 Code, RTT e protocolli di routing

`tc netem` può aumentare ritardo e perdita. Gli effetti attraversano più capitoli:

- ping misura RTT maggiore;
- TCP riduce throughput se il BDP non è coperto;
- CUBIC/BBR reagiscono diversamente;
- i timer OSPF/BGP troppo aggressivi possono scadere;
- BFD può generare falsi positivi se configurato oltre la capacità del percorso;

- le code piene producono bufferbloat e influenzano l'interattività.

13.6 LPM e traffic engineering BGP

Gli attributi BGP decidono tra rotte dello stesso prefisso. Il LPM viene applicato prima: una rotta /24 vince su una /16 indipendentemente da LOCAL_PREF o AS_PATH, perché sono voci per destinazioni diverse nella FIB.

Per questo gli annunci più specifici sono uno strumento potente di traffic engineering inbound.

DOMANDA TIPICA D'ESAME

“Tra una rotta /16 con local preference 200 e una /24 con local preference 100, quale viene usata per un indirizzo nel /24?” La /24, perché il forwarding applica LPM. Gli attributi BGP hanno già selezionato il best path separatamente per ciascun prefisso.

Appendice A. Schede dei protocolli complementari

A.1 ICMP e ICMPv6

ICMP trasporta messaggi di errore e diagnostica per IP. Non è un protocollo applicativo generico e non usa porte TCP/UDP.

Messaggi IPv4 comuni:

- Echo Request/Reply;
- Destination Unreachable;
- Time Exceeded;
- Redirect;
- Parameter Problem.

ICMPv6 svolge anche funzioni indispensabili come NDP e Path MTU Discovery.

Voce	ICMP/ICMPv6
Scopo	Diagnostica, errori e funzioni di controllo IP
Vantaggi	Feedback di rete, ping, traceroute, PMTUD
Svantaggi	Può essere filtrato o abusato; richiede rate limiting
Algoritmo	Messaggi request/reply ed error reporting
Metrica	RTT misurabile con Echo, non metrica di routing
Casi d'uso	Diagnosi, NDP, PMTUD
Domande d'orale	Perché traceroute riceve Time Exceeded; perché non bloccare tutto ICMPv6; differenza tra errore ICMP e porta

A.2 BFD

BFD rileva rapidamente guasti bidirezionali tra due endpoint e notifica i protocolli client.

Voce	BFD
Scopo	Rilevazione rapida di failure
Vantaggi	Tempi sub-secondo, indipendente dal protocollo di routing
Svantaggi	Timer aggressivi consumano CPU e possono causare falsi positivi
Algoritmo	Scambio periodico di control packet e detection multiplier
Metrica	Detection time, non costo di routing

Voce	BFD
Casi d'uso	OSPF, BGP, MPLS, data center
Domande d'orale	Perché non basta ridurre Hello/Hold; come si calcola il detection time; differenza tra rilevazione e convergenza

A.3 SNMP

Voce	SNMP
Scopo	Monitorare e gestire dispositivi
Vantaggi	Diffuso, semplice, MIB standard
Svantaggi	Polling, limiti di sicurezza nelle versioni vecchie, modelli talvolta rigidi
Algoritmo	GET/SET/GETBULK + TRAP/INFORM
Metrica	Non applicabile; trasporta contatori e gauge
Casi d'uso	NMS, inventario, allarmi, contatori interfaccia
Domande d'orale	MIB vs OID; trap vs polling; SNMPv2c vs v3

A.4 NETCONF/YANG

Voce	NETCONF/YANG
Scopo	Configurazione strutturata e modellata
Vantaggi	Validazione, transazioni, automazione, datastore
Svantaggi	Complessità, supporto dipendente dai device
Algoritmo	RPC su datastore; YANG definisce schema e vincoli
Metrica	Non applicabile
Casi d'uso	Automazione enterprise/ISP, orchestrazione
Domande d'orale	YANG è protocollo? running vs candidate; differenza da SNMP

Appendice B. Ricettario di laboratorio Kathará

B.1 Verifica iniziale di un nodo

```
hostname
ip -br link
ip -br addr
ip route
ip -6 route
ip neigh
ss -lntup
```

Domande da porsi:

- l'interfaccia corretta è UP?
- l'indirizzo ha la maschera corretta?
- il next hop è nella stessa rete del link?
- esiste la rotta di ritorno?
- il processo atteso è in ascolto?

B.2 Verifica del forwarding

```
sysctl net.ipv4.ip_forward
sysctl net.ipv6.conf.all.forwarding
```

Cattura contemporanea sui due lati del router:

```
tcpdump -nni eth0 icmp
# altro terminale
tcpdump -nni eth1 icmp
```

Interpretazione:

- pacchetto su ingresso ma non su uscita: routing, forwarding, firewall o TTL;
- pacchetto su uscita ma nessuna risposta: destinazione o percorso di ritorno;
- risposta arriva a router ma non all'host: rotta/ARP sul lato sorgente o firewall.

B.3 Tabella di routing Linux

```
ip route
ip route get 10.0.2.2
ip -6 route get 2001:db8:2::2
```

`ip route get` mostra la decisione effettiva del kernel, inclusi next hop, interfaccia e indirizzo sorgente.

Rotte statiche:

```
ip route add 10.0.2.0/24 via 10.0.1.2
ip route replace default via 10.0.0.254
ip route del 10.0.2.0/24
```

B.4 Diagnosi FRR

```
systemctl status frr
journalctl -u frr --no-pager
vtysh -c 'show running-config'
vtysh -c 'show ip route'
```

Da `vtysh`:

```
show daemons
show interface
show ip route
show ipv6 route
show running-config
```

Protocollo specifico:

```
show ip rip
show ip ospf neighbor
show ip ospf database
show ip bgp summary
show ip bgp neighbors
```

B.5 Catture per protocollo

```
# ARP
tcpdump -ttni eth0 arp

# ICMPv4
tcpdump -nni eth0 icmp

# ICMPv6/NDP
tcpdump -nni eth0 icmp6

# RIP
tcpdump -nni eth0 'udp port 520'

# OSPFv2
tcpdump -nni eth0 'ip proto 89'

# BGP
tcpdump -nni eth0 'tcp port 179'
```

B.6 Test di percorso

```
ping -c 3 <next-hop>
ping -c 3 <destinazione>
traceroute -n <destinazione>
ip route get <destinazione>
```

Ordine corretto:

1. proprio indirizzo;
2. vicino sul link;
3. interfaccia successiva del router;
4. host remoto.

Testare direttamente l'host remoto senza verificare gli hop rende più difficile localizzare il problema.

B.7 Test di prestazioni ripetibile

Server:

```
iperf3 -s -1
```

Client:

```
iperf3 -c 10.0.0.2 -t 20 -0 3 -J > baseline.json
```

- `-0 3`: esclude i primi secondi di warm-up dalle statistiche;
- `-J`: JSON.

Dopo una policy `tc`:

```
iperf3 -c 10.0.0.2 -t 20 -0 3 -J > shaped.json
```

Confrontare:

```
jq '.end.sum_received.bits_per_second' baseline.json shaped.json
```

B.8 Errori Kathará frequenti

Sintomo	Possibile causa	Verifica
Interfaccia assente	<code>lab.conf</code> errato	<code>ip -br link</code> , controllare collision domain
Ping al vicino fallisce	IP/mask/ARP/link	<code>ip addr</code> , <code>ip neigh</code> , <code>tcpdump ARP</code>
Ping remoto fallisce	Rotta o forwarding	<code>ip route get</code> , <code>sysctl forwarding</code>
FRR non parte	daemon disabilitato/config errata	<code>/etc/frr/daemons</code> , <code>journalctl</code>
OSPF senza neighbor	area/timer/MTU/link	Hello con <code>tcpdump</code> , <code>show ip ospf interface</code>
BGP Active	TCP/remote-as/indirizzo	ping, <code>ss</code> , <code>tcpdump 179</code>
BGP Established senza rotte	network/policy/filtro	<code>show ip bgp</code> , <code>running-config</code>
Rotta BGP non installata	next hop irraggiungibile o rotta migliore	<code>show ip bgp</code> , <code>show ip route</code> , <code>ip route</code>
iperf connection refused	server non avviato	<code>ss -lnp</code> , <code>iperf3 -s</code>
tc non cambia nulla	interfaccia/direzione errata	<code>tc -s qdisc show</code> , <code>tcpdump</code>

Appendice C. Errori concettuali da evitare

- “Routing e forwarding sono la stessa cosa.”**
No: routing calcola, forwarding applica.
- “Un router è un host con due schede.”**
Non basta: deve inoltrare pacchetti e avere un control/data plane coerente.
- “ARP trova il MAC della destinazione remota.”**
Trova il MAC del next hop sul link locale.
- “IPv6 non ha bisogno di sicurezza perché non usa NAT.”**
NAT e firewall hanno funzioni diverse.
- “NDP è solo ARP con indirizzi più lunghi.”**
Include router discovery, SLAAC, DAD, NUD e redirect.
- “OSPF invia tutta la tabella ogni 30 minuti.”**
OSPF diffonde LSA e refresh; non funziona come un Distance Vector periodico.
- “DR è il router che inoltra tutto il traffico della LAN.”**
DR coordina adiacenze e LSA; il forwarding dei dati segue la tabella IP.
- “La LSDB è la routing table.”**
LSDB -> SPF -> RIB -> FIB.
- “BGP sceglie sempre il percorso con meno AS.”**
LOCAL_PREF e policy possono prevalere.
- “MED e LOCAL_PREF fanno la stessa cosa.”**
LOCAL_PREF decide l'uscita interna e vince alto; MED suggerisce l'ingresso al vicino e vince basso.
- “Una sessione BGP Established prova che la rete funziona.”**
Prova solo che il peering è attivo; servono rotte valide e forwarding.
- “UDP non ha alcun controllo degli errori.”**
Ha checksum; non ha ritrasmissione e ordinamento integrati.
- “Flow control protegge la rete.”**
Protegge il ricevitore. Congestion control protegge il percorso.
- “Slow Start è lento.”**
La crescita è esponenziale; il nome è storico.
- “Tre duplicate ACK equivalgono a un timeout.”**
Sono segnali diversi e Reno reagisce diversamente.

16. **"QUIC è UDP puro e quindi inaffidabile."**
QUIC implementa affidabilità, sicurezza e congestion control sopra UDP.
17. **"Più buffer significa sempre meno perdita e quindi rete migliore."**
Buffer eccessivi producono bufferbloat.
18. **"Fairness significa dividere sempre la banda in parti uguali."**
Dipende da flusso, utente, classe e funzione di utilità.
19. **"Un asterisco in traceroute indica un link rotto."**
Indica solo assenza della risposta attesa entro il timeout.
20. **"Se ping funziona, ogni servizio funziona."**
Ping non verifica porte, daemon e policy applicative.

Appendice D. Domande per l'esame orale

D.1 Livello rete

1. Perché è stato introdotto il livello rete?
2. Differenza tra forwarding e routing.
3. Differenza tra RIB e FIB.
4. Come funziona il Longest Prefix Match?
5. Perché un prefisso più specifico prevale su una default route?
6. Inoltro diretto e indiretto.
7. Che cosa significa convergenza?
8. Confronto tra Distance Vector, Link State e Path Vector.
9. Perché il flooding non è una famiglia di routing equivalente alle altre?
10. Quali fasi compongono il tempo di convergenza?

D.2 FRR e Kathará

1. Qual è il ruolo di `zebra`?
2. Perché FRR usa daemon separati?
3. Differenza tra database del protocollo, RIB FRR e tabella kernel.
4. Che cosa significa configurazione operativa?
5. Come si rimuove un comando in vtysh?
6. Perché `write memory` può non bastare in Kathará?
7. Come verificherei che `ospfd` sia in esecuzione?
8. Che cosa contiene `lab.conf`?

D.3 Diagnostica

1. Come funziona ping?
2. Perché il primo ping è spesso più lento?
3. Come funziona traceroute con TTL crescente?
4. Perché traceroute usa tre sonde per hop?
5. Che differenza c'è tra capture filter e display filter?
6. Spiega `tcpdump -t -i eth1`.
7. Come distingueresti un problema di routing da un problema applicativo?
8. Che cosa misura `iperf3` in TCP e UDP?
9. Definisci jitter.
10. Che cos'è il BDP?
11. Perché più flussi TCP possono aumentare il throughput aggregato?
12. Che differenza c'è tra shaping e policing?
13. TBF vs HTB vs netem vs fq_codel.

D.4 RIP

1. Come applica Bellman-Ford?
2. Che metrica usa?
3. Perché 16 significa infinito?
4. Descrivi count-to-infinity con un esempio.
5. Split horizon vs poisoned reverse.
6. Che cosa sono triggered update e route poisoning?
7. RIPv1 vs RIPv2.
8. Perché RIPv2 usa 224.0.0.9?
9. Che effetto ha redistribute connected?
10. Come origineresti una default route?

D.5 IPv6, ARP e NDP

1. Perché IPv6 usa 128 bit?
2. Regole di compressione.
3. Global unicast, link-local, ULA, multicast e anycast.
4. Perché IPv6 non usa broadcast?
5. Che ruolo ha il /64?
6. Che cos'è lo scope di un link-local?
7. Perché ARP non esiste in IPv6?
8. Messaggi RS, RA, NS e NA.
9. Che cosa sono SLAAC, DAD e NUD?
10. Descrivi una transazione ARP.
11. Perché la ARP Request è broadcast e la Reply è unicast?
12. Gratuitous ARP e Proxy ARP.
13. Perché non bisogna bloccare tutto ICMPv6?

D.6 OSPF

1. Descrivi il ciclo completo di OSPF.
2. Che cos'è una LSDB?
3. I cinque tipi di pacchetto.
4. LSA vs LSU.
5. Stati di un'adiacenza.
6. Perché due router possono fermarsi in 2-Way?
7. Perché esistono DR e BDR?
8. Come avviene l'elezione? È preemptive?
9. Come viene calcolato il cost?
10. Perché serve la reference bandwidth?
11. Area 0, ABR e ASBR.
12. Stub area e NSSA.
13. LSA tipo 1, 2, 3, 4, 5 e 7.
14. Intra-area, inter-area, E1 ed E2.
15. Che cosa fa passive-interface?
16. Come funziona ECMP?
17. Timer Hello/Dead e ruolo di BFD.
18. Perché una LSDB coerente riduce i loop?
19. OSPF vs RIP.

D.7 SDN e gestione

1. Data plane vs control plane.

2. Per-router vs logicamente centralizzato.
3. Controller, northbound e southbound API.
4. Match-action in OpenFlow.
5. Che cos'è il traffic engineering?
6. Manager, agent, MIB e OID in SNMP.
7. Trap vs polling.
8. NETCONF vs YANG.
9. Che cosa sono i datastore running e candidate?
10. Perché la telemetria streaming differisce dal polling?

D.8 BGP

1. Che cos'è un AS?
2. IGP vs EGP.
3. Provider-customer, peering e transito.
4. Regola no-free-transit e valley-free.
5. Stub AS e multi-homing.
6. Hot Potato Routing.
7. eBGP vs iBGP.
8. Perché BGP usa TCP?
9. Stati della FSM.
10. Messaggi OPEN, UPDATE, KEEPALIVE, NOTIFICATION.
11. NLRI e attributi.
12. AS_PATH e prevenzione dei loop.
13. LOCAL_PREF vs MED.
14. NEXT_HOP e recursive lookup.
15. Origin e community.
16. Processo di best path.
17. Perché l'ordine esatto è implementation-specific?
18. Problema del full mesh iBGP.
19. Route reflector.
20. `next-hop-self`.
21. Prefix-list e deny implicit.
22. AS-path access-list.
23. Route-map match/set.
24. `network` e import-check in FRR.
25. Perché eBGP può richiedere policy esplicite?
26. Default route per stub AS.
27. Traffic engineering inbound vs outbound.
28. AS-path prepending, MED e community.
29. LPM e annunci più specifici.
30. Prefix hijack, route leak e RPKI.
31. OSPF vs BGP.

D.9 Data center

1. Perché leaf-spine?
2. Clos e Fat-Tree.
3. Che cosa significa traffico east-west?
4. Come funziona ECMP?
5. Perché hashing per flusso?
6. Perché usare BGP come underlay?
7. ASN privati.

8. `maximum-paths` e `multipath-relax`.
9. Perché timer aggressivi sono rischiosi?
10. Perché traceroute tradizionale è ambiguo con ECMP?

D.10 Trasporto e affidabilità

1. Perché serve il livello di trasporto se esiste IP?
2. Multiplexing e demultiplexing.
3. 2-tuple UDP e 4-tuple TCP.
4. Proprietà di UDP.
5. Proprietà di TCP.
6. Perché TCP è un byte stream?
7. Checksum, ACK/NAK, numeri di sequenza e timer.
8. Perché Stop-and-Wait è inefficiente?
9. Definisci sliding window.
10. Go-Back-N vs Selective Repeat.
11. ACK cumulativi e SACK.
12. Three-Way Handshake.
13. Flow control vs congestion control.
14. `rwnd`, `cwnd` e flight size.
15. Perché RTO deve adattarsi al RTT?

D.11 Congestione, QUIC e fairness

1. Che cos'è il collasso da congestione?
2. Come funziona il self-clocking?
3. Slow Start e Congestion Avoidance.
4. Timeout vs tre duplicate ACK.
5. Tahoe vs Reno.
6. AIMD e convergenza verso fairness.
7. Perché i flussi con RTT diverso possono essere iniqui?
8. Come funziona CUBIC?
9. Regioni concava, plateau e convessa.
10. Che cosa stima BBR?
11. Perché QUIC usa UDP?
12. Vantaggi del setup QUIC.
13. 0-RTT e rischio di replay.
14. Stream indipendenti e head-of-line blocking.
15. Connection migration.
16. Max-min fairness.
17. Water filling e bottleneck link.
18. Proportional fairness.
19. Utility fairness.
20. Tail drop e bufferbloat.
21. RED vs CoDel/fq_codel.
22. Come funziona ECN?
23. DCTCP.
24. Che cosa significa TCP-friendly?
25. Perché una coda prioritaria assoluta può essere pericolosa?

Appendice E. Glossario essenziale

Termine	Definizione
ABR	Area Border Router OSPF
ACK	Conferma positiva di ricezione
AQM	Gestione attiva delle code
AS	Autonomous System
ASBR	Router che redistribuisce rotte esterne in OSPF
ASN	Numero identificativo di un AS
BDP	Banda moltiplicata per RTT; dati necessari in volo
BFD	Rilevazione rapida bidirezionale dei guasti
BGP	Protocollo Path Vector inter-dominio
BDR	Backup Designated Router
BBR	Congestion control model-based basato su banda e RTT
Bufferbloat	Latenza elevata dovuta a code persistenti
CIDR	Indirizzamento classless con lunghezza di prefisso
Clos	Architettura multistadio con molti percorsi
Community	Etichetta BGP usata dalle policy
Control plane	Calcolo e distribuzione delle regole di inoltro
Cost	Metrica OSPF
CUBIC	Algoritmo TCP con crescita cubica della cwnd
cwnd	Congestion window del mittente
Data plane	Inoltro dei pacchetti
DCTCP	TCP per data center basato sulla frazione di marcature ECN
DR	Designated Router OSPF
ECMP	Uso contemporaneo di più percorsi a costo uguale
ECN	Notifica esplicita di congestione tramite marcatura
eBGP	BGP tra AS diversi
FIB	Tabella ottimizzata usata per il forwarding
Flow control	Protezione del ricevitore
FRR	Suite di routing open source per Linux
GBN	Go-Back-N
Hold Time	Tempo BGP massimo senza messaggi validi prima di chiudere il peer
HTB	Qdisc gerarchica per classi e banda
iBGP	BGP interno a un AS
ICMP	Protocollo di controllo e diagnostica IP
IGP	Protocollo di routing interno a un AS
LPM	Longest Prefix Match
LSA	Link State Advertisement
LSDB	Link State Database
MED	Multi-Exit Discriminator BGP

Termine	Definizione
MIB	Modello gerarchico degli oggetti SNMP
MSS	Maximum Segment Size TCP
NDP	Neighbor Discovery Protocol IPv6
NETCONF	Protocollo RPC di configurazione di rete
NEXT_HOP	Attributo BGP che indica il prossimo router
NUD	Neighbor Unreachability Detection
OSPF	IGP Link State basato su SPF
Path Vector	Modello che annuncia il percorso dei domini
Prefix-list	Filtro ordinato di prefissi
QUIC	Trasporto sicuro sopra UDP, base di HTTP/3
RED	Random Early Detection
RIB	Base informativa delle rotte
RIP	IGP Distance Vector a hop count
RPKI	Infrastruttura per validare l'origine dei prefissi
RTT	Round Trip Time
rwnd	Receive window pubblicata dal ricevitore
SDN	Software Defined Networking
SLAAC	Autoconfigurazione stateless IPv6
SPF	Shortest Path First
ssthresh	Soglia tra Slow Start e Congestion Avoidance
TBF	Token Bucket Filter
tc	Linux Traffic Control
TCP	Trasporto affidabile a flusso di byte
UDP	Trasporto best-effort a datagrammi
vytysh	Shell unificata di FRRouting
YANG	Linguaggio di modellazione dei dati di rete
Zebra	Daemon FRR che coordina le rotte e il kernel

Checklist finale prima dell'esame

Devi riuscire a spiegare senza appunti:

- la catena **LSDB -> SPF -> RIB -> FIB -> forwarding**;
- la differenza **RIP / OSPF / BGP**;
- la catena **IP destination -> LPM -> next hop -> ARP/NDP -> frame**;
- il ruolo di **zebra** in FRR;
- il metodo di diagnosi con **ip, ping, traceroute, tcpdump, Wireshark**;
- un test completo con **iperf3 + tc/netem**;
- la differenza **flow control / congestion control**;
- la progressione **checksum -> ACK -> SEQ -> timer -> sliding window**;
- **Go-Back-N vs Selective Repeat**;
- **Slow Start, Congestion Avoidance, Fast Retransmit, Fast Recovery**;
- **Reno vs CUBIC vs BBR**;
- i vantaggi di **QUIC**;

- **max-min, proportional e utility fairness;**
- **tail drop, RED, fq_codel, ECN e DCTCP.**

STRATEGIA PER L'ORALE

Parti sempre dal problema, poi descrivi il meccanismo, infine vantaggi, limiti e un'osservazione di laboratorio. Una risposta strutturata vale più di una sequenza di definizioni isolate.